РУКОВОДСТВО
ПО ПРОГРАММИРОВАНИЮ
МИКРОКАЛЬКУЛЯТОРА
"ЭЛЕКТРОНИКА МК-90"

МАТЕМАТИЧЕСКОЕ ОБЕСПЕЧЕНИЕ

БЕЙСИК V2.1

БРЕСТ
1990

Содержание

	лист
1. Общие сведения	3
1.1. Символы языка	3
1.2. Принятые условные обозначения	3
1.3. Строки	3
2. Константы	4
3. Переменные	4
4. Выражения	5
5. Команды Бейсика микрокалькулятора	7
5.1. Команды непосредственного режима	7
5.1.1. Команда AUTO	8
5.1.2. Команда BLOAD	9
5.1.3. Команда BSAVE	10
5.1.4. Команда CONT	11
5.1.5. Команда DATE	12
5.1.6. Команда DELETE	13
5.1.7. Команда DEV	14
5.1.8. Команда EDIT	15
5.1.9. Команда FILES	17
5.1.10. Команда INIT	18
5.1.11. Команда KILL	19
5.1.12. Команда LIST	20
5.1.13. Команда LOAD	22
5.1.14. Команда MEM	23
5.1.15. Команда NAME/AS	24
5.1.16. Команда NEW	25
5.1.17. Команда PATCH	26
5.1.18. Команда RENUM	27
5.1.19. Команда RUN	28
5.1.20. Команда SAVE	29
5.1.21. Команда SYSTEM	30
5.1.22. Команда TIME	31
5.1.23. Команда USR	32
5.2. Команды программного режима	33
5.2.1. Оператор CALL	34
5.2.2. Оператор CLOSE	38
5.2.3. Оператор CLS	39
5.2.4. Оператор DEF FN	40
5.2.5. Оператор DIM	43
5.2.6. Оператор DIS	46
5.2.7. Оператор DRAW	47
5.2.8. Оператор END	53
5.2.9. Оператор FOR-TO-STEP/NEXT	54
5.2.10. Оператор GET	55
5.2.11. Оператор GOSUB,RETURN	56
5.2.12. Оператор GOTO	58
5.2.13. Оператор IF/THEN	60
5.2.14. Оператор INPUT	62
5.2.15. Оператор LET	63
5.2.16. Оператор LOCATE	64
5.2.17. Оператор OPEN	65
5.2.18. Оператор PAINT	66

5.2.19. Оператор PLAY	67
5.2.20. Оператор PRINT	68
5.2.21. Оператор PUT	71
5.2.22. Оператор RANDOMIZE	72
5.2.23. Оператор READ,DATA,RESTORE	74
5.2.24. Оператор REM	77
5.2.25. Оператор STOP	78
5.2.26. Оператор TROFF	79
5.2.27. Оператор TRON	80
5.2.28. Оператор WAIT	81
5.3. Встроенные функции	82
5.3.1. Функция ABS	83
5.3.2. Функция ATN	84
5.3.3. Функция SIN,COS	85
5.3.4. Функция EXP	86
5.3.5. Функция INC	87
5.3.6. Функция INT	88
5.3.7. Функция LOG	89
5.3.8. Функция PI	91
5.3.9. Функция RND	92
5.3.10. Функция SGN	93
5.3.11. Функция SQR	94
Приложение 1. Схема клавиатуры для работы в функциональном режиме	95
Приложение 2. Команды Бейсика микрокалькулятора	96
Приложение 3. Сообщения об ошибках	111
Приложение 4. Примеры программ	116
Приложение 5. Таблица масок для оператора PAINT	127
Приложение 6. Таблицы кодов символов	129

1. Общие сведения.

1.1. Символы языка.

Для записи программ на языке Бейсик используются следующие символы:
а) буквы латинского алфавита (A-Z);
б) цифры (0-9);
в) специальные символы:
пробел * / + - ^ = < > . , ; : ( ) "

Оператор печати и REM допускают использование букв русского алфавита и любых символов.

Чтобы получить буквы русского алфавита необходимо нажать клавишу Iрус/латI. После того как информация на русском языке набрана, еще раз нажимается клавиша Iрус/латI.

1.2. Принятые условные обозначения.

В примерах, поясняющих синтаксис Бейсика, использованы следующие условные обозначения:
а) < > (угловые скобки) указывают на необходимые элементы описываемого оператора или команды;
б) [ ] (квадратные скобки) указывают на необязательность присутствия элемента в описываемом операторе или команде.

Элементы, напечатанные строчными буквами и заключенные в угловые скобки задаются пользователем в соответствии с правилами, объясняемыми в тексте. Элементы, напечатанные заглавными буквами, должны иметь тот же вид.

1.3. Строки.

Программа на языке Бейсик представляет собой последовательность строк. Каждая строка программы начинается с номера и заканчивается символом ВК . Номер строки должен быть целым числом > 0. Операторы выполняются в порядке возрастания номеров их строк. За каждым номером строки следует оператор. Все операторы Бейсика и выражения должны быть не более 80 символов. Однако, на одной строке (длиной 80 символов) может быть записано несколько операторов. В этом случае они разделяются между собой двоеточием. Пробелы могут встречаться в любом месте программы, не влияя на ее выполнение.

2. Константы и точность вычисления

В языке Бейсик определены целые и вещественные константы. Вещественные константы записываются в основной и экспоненциальной форме.
Например, число 2340.0 на языке Бейсик может быть записано как целое число 2340 или вещественное 2340.0 (основная форма), 23.4E2 или 2.34E3 (экспоненциальная форма).

Для использования чисел, имеющих экспоненциальное представление, существуют следующие правила:
- порядок может не иметь знака, если он положителен (1234.56E3). отрицательный порядок должен иметь знак (1234.56E-3);
- порядок должен находиться в интервале от - 980 до +980;

Результаты вычислений выводятся на экран ЖКИ в виде целых или десятичных чисел, если для представления результата достаточно девяти позиций, т.е. семь цифр, знак (пробел в случае положительного результата) и десятичная точка. В противном случае результат выводится на экран ЖКИ в экспоненциальном представлении и занимает максимум пятнадцать позиций, т.е. знак (если результат положительный - пробел), десятичная точка, семь значащих цифр мантиссы, символ E, знак порядка (минус или пробел) и порядок, имеющий до четырех цифр.

Ниже приведены некоторые вводимые пользователем значения с эквивалентами, печатаемыми Бейсиком.

Вводимое значение	Значение, печатаемое Бейсиком
.01	.01
9.9000E-3	.0099
999999	999999
1.000000E6	1000000
2.718281828459045	2.718282
0.0000000036218	.36218E-9

Бейсик точен по крайней мере до семи цифр с точностью до единицы в седьмом разряде.
Поэтому число 4.0000000225 является эквивалентом 4.0.

3. Переменные

Переменная есть величина, значение которой может изменяться в процессе выполнения программы. Переменная обозначается одной буквой или одной буквой (заглавной латинской) с последующей за ней цифрой.

Например:

1, B3, C8

Так же как и константы, переменные могут быть целыми и вещественными.

4. Выражения

Арифметическое выражение на языке Бейсик состоит из констант, переменных и функций, соединенных знаками арифметических операций. Для указания порядка выполнения этих действий можно использовать круглые скобки. Для обозначения арифметических операций используются следующие знаки:

^ возведение в степень
+,- сложение, вычитание
*,/ умножение, деление

Примечание: печатный знак < ^ > в руководстве означает знак < - > на экране ЖКИ микрокалькулятора.

Ограничения области вычислений:

1) деление на 0 приводит к ошибке;
2) при переполнении , когда результат не укладывается в диапазон вычислений , возникает ошибка.

Допустимые степени:

X^Y

Если Y - любое целое число, то
X - любое действительное число.
Если Y - любое действительное число, то
X - действительное положительное число.
Ошибка при X<=0.

Примеры:

1.	0.5^0	1
2.	-0.5^0	-1
3.	(-0.5)^0	ошибка
4.	0.5^2	0.25
5.	0.5^(-2)	4
6.	(-0.5)^-2	ошибка
7.	0.5^0.5	0.7071068
8.	(-0.5)^0.5	ошибка
9.	2^(-0.5)	0.7071068
10.	(2)^-0.5	ошибка

Приоритет последовательности вычислений:

1) Элементы в круглых скобках.
2) Функции (встроенные функции (см. п.6.3.3) и функции пользователя (см. п. 6.3.2.4))
3) Возведение в степень (^)
4) Умножение (*) и деление (/)
5) Сложение (+), вычитание (-)
6) Операторы сравнения (=,),< и.т.д.).

Операторы сравнения

Операторы сравнения могут быть использованы только с применением оператора IF (см.п 6.3.2.13)

=	равно
<>,><	не равно
<	меньше
>	больше
=>, >=	равно или больше
=<, <=	равно или меньше

Примеры: a+b<>0. результат сложения a и b не равен нулю.

5. Команды Бейсика версии V2.1 микрокалькулятора

5.1. Команды непоосредственного режима

Некоторые команды Бейсика могут вводиться в строке без номера. В этом случае выполнение команды осуществляется после нажатия клавиши ВК в конце строки. Результат выполнения хранится для последующего использования, но команда теряется. Этот режим называется непосредственным. Непосредственный режим необходим при редактировании и отладке програми, записанных в программном режиме.
В приводимых форматах команд квадратные скобки [ ] означают возможность или необязательность присутствия информации.

Краткое описание команд Бейсика микрокалькулятора приведено в Приложении 2. Сообщения об ошибках при вводе команд приведены в Приложении 3.

5.1.1. Команда A U T O

Функция: переводит Бейсик в режим автоматичеокой нумерации строк во время ввода программы.
Формат: AUTO [<номер строки>][,[<шаг>]]

Выполнение этой команды приводит к генерации номеров строк после каждого нажатия клавиши ВК в конце очередной программной строки. Нумерация начинается с <номера строки>, а последующие номера строк отличаются на величину, заданную <шагом>. Если <номер строки> или <шаг> не заданы, то значением по умолчанию принимается 10. Если команда имеет следующий вид:

AUTO <номер строки>,

по умолчанию <шаг> принимается равным шагу, указанному в предыдущем операторе AUTO.
Прекращение автонумерации осуществляется по нажатию клавиш СУ/Р.

Пример :

```
AUTO 100,50
```

Генерация номеров строк с шагом 50 и начальным номером 100. (100,150,200,...)

Пример :

```
AUTO 200,
```

Генерация номеров строк с шагом 50 и начальным номером 200. (200,250,300,...)

Минимальный номер строки и минимальный шаг равны 1. Максимальный номер строки равен 8191. Размер строки не должен превышать 80 символов (4 строки экрана ЖКИ).

5.1.2. Команда B L O A D

Функция: считывает из файла, размещенного на СМП, двоичную информацию и заносит ее в память.
Формат: BLOAD "<спецификация файла>"[<стартовый адрес>][,<длина>]

Спецификация файла состоит из следующих элементов:
[<имя устройства>:]<имя файла>[.<тип>]

Из файла с заданным именем, размещенного на СМП, считывается двоичная информация и заносится в память по указанному стартовому адресу. При отсутствии параметров в команде BLOAD на экран выводятся запросы на подтверждение адреса и длины, которые были указаны при записи этого файла на СМП командой BSAVE.

адрес ...?
длина:

При ответе 'Y ' информация заносится в память по указанному в запросе адресу и указанной длины. Для отказа нужно нажать любую другую клавишу и повторить ввод команды BLOAD с указанием нового адреса, новой длины.

5.1.3. Команда B S A V E

Функция: записывает на СМП файл с двоичной информацией.
Формат: BSAVE "<спецификация файла>"<стартовый адрес>, <длина>

Спецификация файла состоит из следующих элементов:
[<имя устройства>:]<имя файла>[.<тип>]

По умолчанию параметр <тип> принимает значение BIN. Двоичная информация из области памяти, начиная со стартового адреса заданной длины запоминается на СМП в файле с указанным именем.

5.1.4. Команда C O N T

Функция: продолжает выполнение программы после останова.
Формат: CONT

Команда CONT позволяет продолжить выполнение программы после останова, вызванного оператором STOP, начиная с очередного оператора, то есть с оператора, стоящего за оператором STOP.

5.1.5. Команда D A T E

Функция: устанавливает или сообщает текущую дату
Формат: DATE [<число>-<месяц>-<год> <день недели>]

<число>-число месяца,принимает значения от 1 до 31
<месяц>-задается первыми тремя буквами английского названия месяца
Например: март-MAR, июнь-JUN и т.д.
<год>- задается двумя последними цифрами года
Например: 89,90,91 и т.д.
<день недели>-задается первыми тремя буквами английского названия дня недели
Например: понедельник-MON,воскресенье-SUN и т.д.

При указании параметров в операторе DATE устанавливается текущая дата. DATE без параметров выводит текущую дату на экран, если она предварительно была установлена. Если DATE не выводит дату или выводит неверно, значит предварительно она не была установлена. Полное отключение питания сбрасывает ее.

Пример: DATE 25-SEP-90 SUN
задана дата 25 сентября 1990 года воскресенье

Названия месяцев:
январь - JAN
февраль - FEB
март - MAR
апрель - APR
май - MAY
июнь - JUN
июль - JUL
август - AUG
сентябрь - SEP
октябрь - OCT
ноябрь - NOV
декабрь - DEC

Названия дней недели:
понедельник - MON
вторник - TUE
среда - WEN
четверг - THU
пятница - FRI
суббота - SAT
воскресенье - SUN

5.1.6. Команда D E L E T E

Функция: стирает строку с указанным номером
Формат: DELETE <номер строки N1>[,<номер строки N2>]

N1 - номер первой строки, которую необходимо удалить
N2 - номер последней строки
Команда DELETE должна иметь аргумент.

Если в команде DELETE задается только один номер строки, стирается соответствующая строка. Если даны два номера, разделенные запятой, стираются соответствующие две строки и все строки между ними.

Если в команде DELETE начальный номер строки N1 больше конечного N2 - Бейсик отвечает готов, но ни одна строка не стирается.

Пример:
DELETE 54
Стирается строка 54
DELETE 56,876
Стираются строки с 56 по876 включительно
DELETE 1,8191 стираются все строки программы.

5.1.7. Команда D E V

Функция: устанавливает номер рабочего СМП
Формат: DEV ["<имя устройства>:"]

Если <имя устройства> не задано, то в качестве рабочего выбирается СМП0.
Использование команды DEV позволяет не указывать <имя устройства> в <спецификации файла> во всех командах, работающих с СМП.

Пример1:
DEV "SM1:"
Устанавливает в качестве рабочего СМП1.
Пример2:
DEV
Устанавливает в качестве рабочего СМП0.

5.1.8. Команда E D I T

Функция: переводит Бейсик в режим редактирования программы, расположенной в памяти машины

Формат: EDIT [<номер строки>]

Команда EDIT используется в непосредственном режиме для перевода Бейсика в режим редактирования строки. Если параметр <номер строки> не задан, для редактирования выбирается первая строка из буфера команд.

В режиме редактирования используются следующие управляющие клавиши:

I<--I - перемещение маркера на позицию влево;

I-->I - перемещение маркера на позицию вправо;

I ! I - переход к предыдущей строке оператора;

I ! I - переход к следующей строке оператора;

I1-->I - перевод редактора в режим вставки-замены

(I ! I на нижнем регистре);

I<--II - смыкание строки влево к позиции маркера

(I ! I на нижнем регистре;

I ! I - переход к предыдущему оператору в буфере команд;

I ! I - переход к следующему оператору в буфере команд;

I Cy I A I- перемещение маркера в начало редактируемой строки;

I Cy I B I- перемещение маркера в начало предыдущего слова строки;

I Cy I E I- удаление символов строки, начиная с текущей позиции маркера до конца строки;

I Cy I F I- перемещение маркера в начало следующего слова строки;

I Cy I X I- установка маркера за последним символом строки;

I Cy I H.I- вызов подсказки

I BK I - запись отредактированной строки в буфер команд и выход в командный режим.

При редактировании текстовой информации, на экран выводятся обозначения русского и латинского регистров:

S - русский регистр

S - латинский регистр.

Переход в режим редактирования сопровождается очисткой экрана и выводом следующей служебной строки:

```
I edit  зам    вр латI
```

в которой отражаются режимы: верхний/нижний, латинский/русский, а также работа в режиме вставки/замены.

5.1.9. Команда F I L E S

Функция: распечатывает имена файлов с СМП, заданных <спецификацией файла>.

Формат: FILES ["<спецификация файла>"]

Если <спецификация файла> не указана, по этой команде печатается список всех файлов, расположенных на СМП, выбранном в качестве рабочего.

Если <спецификация файла> присутствует, то она состоит из следующих элементов:

[<имя устройства>:][<имя файла><.тип>]

В этом случае команда FILES распечатывает список всех файлов .
Кроме имен файлов, во втором и третьем столбцах списка печатаются соответственно:длина файла в блоках по 512 байт и начальный номер блока этого файла на СМП.
Список файлов разбивается и выводится на экран по страницам после нажатия клавиши ВК .

Пример1:

```
FILES    "SM1:"
```

Распечатывает список всех файлов, расположенных на СМП1.

Пример2:

```
FILES
```

Распечатывает список всех файлов, расположенных на рабочем СМП.

При наборе командной строки могут быть ошибки - отсутствие кавычек, двоеточия или неправильно задано символическое имя устройства, а также набрано больше шести символов в имени файла.
Во всех этих случаях выдается одно сообщение об ошибке.

5.1.10. Команда I N I T

Функция: стирание всех файлов с СМП и его разметка.

Формат: INIT ["<имя устройства>:"]

Если <имя устройства> не указано, то стирается и размечается рабочий СМП . Вся информация с СМП исчезает.
После задания команды INIT выдается запрос на подтверждение выполнения операции стирания:

" вы уверены? ".

Для продолжения выполнения команды необходимо набрать "Y". Нажатие любого другого символа приводит к отмене команды.

Примеры:

```
INIT "SM0:"
INIT
```

Примечание: при выполнении команды INIT ,также как и всех команд, предназначенных для работы с СМП снятие по нажатию СУ/Р не выполняется.

5.1.11. Команда K I L L

Функция: удаляет файлы с СМП
Формат: KILL "<спецификация файла>"

Команда KILL может использоваться для удаления файлов любых типов с СМП. Для удаления файла обязательно указывается полная спецификация файла, если удаляемый файл расположен на устройстве, отличном от рабочего. В противном случае достаточно указать имя и тип файла.

После задания команды KILL выводится запрос на подтверждение удаления:

" вы уверены? ".

При задании " Y " команда выполняется, нажатие любой другой клавиши прекращает ее выполнение.

Пример:

KILL "SM1:GRAPH.BAS"

Удаляется файл GRAPH.BAS с устройства SM1.

5.1.12. Команда L I S T

Функция: печатает на экране строки программы.
Форматы:

LIST [<номер строки N1>[,<номер строки N2>]]
LIST "<спецификация файла>"[<номер строки>]

Программа может быть либо загружена в память, либо находиться в файле на СМП. Во втором случае текст программы печатается без загрузки ее в память.

1. Печать текста программы, загруженной в память.

Формат: LIST [<номер строки N1>[,<номер строки N2>]]

Для получения текста программы на экране нужно задать команду LIST без параметров. По окончании печати Бейсик выйдет на "Готов".

Для печати одной строки нужно в команде LIST указать номер этой строки. Для печати ряде строк в команде LIST следует указать номера начальной и конечной строк; в этом случае, кроме указанных строк, будут печататься все строки между ними.

Если между указанными номерами находится более 7 строк программы, то на экран выводится сразу 7 строк, а далее по одной строке после каждого нажатия клавиши ВК.

При задании

LIST ,<номер строки N2>

выполняется печать текста программы от строки с начальным номером до строки с номером N2.

При задании

LIST <номер строки N1>,

выполняется печать текста программы от строки с номером N1 до конца программы.

Примечание. При вызове командой LIST несуществующего номера строки (строк)на экран выводится ближайшая к ней (ним) существующая строка (строки).

Пример.

LIST 37

Печатается строка 37

LIST 37,126

Печатаются строки с 37 по 126 включительно

LIST

Печатается вся программа начиная со строки с меньшим номером.

2. Печать текста программы из файла, имеющего ASCII-формат и размещенного на СМП.

Формат: LIST "<спецификация файла>"[<номер строки>]

Спецификация файла состоит из следующих элементов:
[<имя устройства>:]<имя файла>[.<тип файла>]

Для получения текста всей программы нужно задать команду LIST "<спецификация файла>". При задании номера пе-

чатаются строки программы, начиная с указанной и до последней строки. Каждая последующая строка, кроме первой, выводится по нажатию клавиши ВК.

Преждевременно завершить выполнение этой команды можно нажав клавиши СУ/Р.

По окончании команды Бейсик печатает на экране "Готов".

Пример:

```
LIST "SM1:EX1.BAS"
```

Печатается текст всей программы.

```
LIST "EX2.BAS"100
```

Печатается текст программы со строки 100.

5.1.13. Команда L O A D

Функция: загружает в память программу, записанную на СМП

Формат: LOAD "<спецификация файла>"[,A][,R]

Спецификация файла состоит из имени устройства, имени и типа файла. При отсутствии имени устройства подразумевается рабочий СМП. По умолчанию тип файла принимает значение ".BAS", если задан параметр A или ".BAC", если параметр A отсутствует.

Если задан параметр A, то в память загружается программа в коде ASCII. Отсутствие параметра A означает, что в память загружается программа во внутреннем формате.

Если команда LOAD выполняется без задания "R", то после загрузки программы Бейсик переходит в командный режим. Если "R" задано, то как только загрузка программы завершится, сразу же начинается ее выполнение. Необходимо помнить, что в результате работы команды LOAD все переменные очищаются, а любая программа, находившаяся до этого в буфере команд, уничтожается.

В Бейсик V1.0 сохранение программ на СМП выполнялось только в коде ASCII. Чтобы загрузить программу, записанную на СМП в Бейсике V1.0 необходимо обязательно указывать параметр "A".

Пример:

```
LOAD "SM1:PROG1.BAS",A
```

5.1.14. Команда M E M

Функция: выдает распределение памяти в текущий момент

Формат: MEM

Пример: MEM

Выдает распределение памяти в следующем виде:

```
распределение памяти (в байтах)
программа:  1
свободно:  12247
```

эта команда дает информцию об об'еме набранного текста программы, при этом необходимо учитывать, что об'ем памяти, занимаемый переменной или элементом массива равен 6 байт.

5.1.15. Команда N A M E / A S

Функция: изменяет имена файлов на СМП

Формат: NAME "<спецификация старого файла>" AS "<спецификация нового файла>"

Как <спецификация старого файла>, так и <спецификация нового файла> задаются именем устройства, именем и типом файла. Имя устройства допускается опускать, если это рабочий СМП.
Имя файла, указанное в <спецификации старого файла>, должно быть именем существующего файла.Если имя нового файла уже сушествует, то файл с таким именем будет удален с СМП, а его имя присвоится файлу со спецификацией старого файла. Эта команда изменяет только имя указанного файла, но не переписывает его в другую область СМП.

Пример:

```
FILES "SM1:"
PROG1 .BAS      2   4
PROG3 .BAS      5   6
Готов
NAME "SM1:PROG1.BAS" AS "SM1:PROG2.BAS"
Готов
FILES "SM1:"
PROG2 .BAS      2   4
PROG3 .BAS      5   6
Готов
```

5.1.16. Команда N E W

Функция: удаляет из памяти все программные строки, переменные и массивы.
Формат: NEW

Команда NEW позволяет пользователю удалить из памяти все программные строки,переменные и массивы.

5.1.17. Команда P A T C H

Функция: позволяет проверять и изменять содержимое любой ячейки программы пользователя в машинных кодах.
Формат: PATCH

Для работы с функцией PATCH используются следующие операторы и команды:

XXXXXX- аргумент, представляющий адрес ячейки, который интерпретируется как 6-значное восьмеричное число;

/ или A/ - оператор вывода восьмеричных слов.Повторно открывает и печатает содержимое последнего открываемого слова.Значение A воспринимается как адрес слова, которое должно быть открыто.

\ или A\ - оператор вывода восьмеричных байтов. Повторно открывает и печатает содержимое последнего открываемого байта.Значение A воспринимается, как адрес байта, который должен быть открыт.

<ВК> или M <ВК> - закрывает текущую открытую ячейку.Значение M заменяет содержимое текущей открытой ячейки перед тем, как она будет закрыта.

или M - закрывает текущую открытую ячейку,открывает следующую последовательную ячейку.Значение M заменяет содержимое текущей открытой ячейки перед тем,как открыть следующую.

или M - закрывает текущую открытую ячейку и открывает предыдущую ячейку.Значение M заменяет содержимое текущей открытой ячейки перед тем,как открыть предыдущую.

@ или M @- интерпретирует содержимое текущей открытой ячейки, как абсолютный адрес.Закрывает текущую открытую ячейку и открывает ячейку по этому абсолютному адресу.Значение M заменяет содержимое текущей открытой ячейки перед тем,как она будет закрыта и использоваться как абсолютный адрес.

A Б - запуск программы пользователя с текущего адреса.

E - окончание работы с PATCH.выход Бейсика на 'Готов'.

Для написания программ в машинных кодах используется система команд ЭВМ "Электроника 60".
Пример: приведен в операторе 6.3.2.1. оператор CALL

5.1.18. Команда R E N U M

Функция: перенумерация строк программы.
Формат: RENUM <номер строки X1>,<номер строки X2>,
<приращение>

RENUM позволяет присваивать новые номера одного порядка целому блоку строк программы, начиная со строки с заданным номером и кончая последней строкой,находящейся в памяти.
X1-первый(наименьший) новый номер строки;
X2-старый номер,с которого начинать перенумерацию строк;
<приращение>-разность между двумя соседними новыми номерами;
Если X1=0, то по умолчанию новый номер равен 10;
X2=0, то перенумерация будет начинаться с первого номера строки программы;
<приращение>=0, то шаг перенумерации будет равен 10.
В случае,если какой-то из параметров отсутствует,на экран ЖКИ выдается сообщение об ошибке.

Номер строки,с которой начинается перенумерация,должен задаваться так,чтобы строка с указанным номером действительно имелась в програнне.В противном случае перенумерация выполнять-ся не будет,а появиться сообщение об ошибке.

Операторы GOTO, GOSUB, IF-THEN всегда содержат ссылки на номера различных строк программы;команда RENUM изменяет и эти номера. Замена не производится, если это ссылка на номер строки, не входящий в перенумеруемый блок программных строк. Если в ходе такой замены встречается ссылка на несу-ществующую строку,то выдается сообщение "недопустимая ссылка" и указывается номер отсутствующей строки и строки,в которой обнаружена ссылка на нее,а затем процесс перенумерации строк продолжается.

Пример:
```
 10 PRINT:GOTO 20
 20 GOTO 30A
 30 GOTO 5:GOSUB 40
 40 IF X=1 THEN GOSUB 50
 50 IF X=2 GOTO 20
 60 IF X=3 THEN 10
 70 IF X=4 THEN GOTO 30

RENUM 1,10,100

  1  PRINT:GOTO 101
101 GOTO 30A
201 GOTO 5:GOSUB 301
301 IF X=1 THEN GOSUB 401
401 IF X=2 THEN GOTO 101
501 IF X=3 THEN 1
601 IF X=4 THEN GOTO 201
НЕДОПУСТИМАЯ ССЫЛКА  5-201
```
В операторе 101 GOTO 30A выражение не обрабатывается.

5.1.19. Команда R U N

Функция: осуществляет запуск программы
Формат: RUN

По команде RUN запускается программа в программном режиме.

При использовании команды RUN для повторного прохождения программы все значения переменных стираются из памяти. Если в программу включена функция RND(x), то ее начальное значение восстанавливается.

Прекратить выполнение программы можно последовательным нажатием клавиш CУ/P .

5.1.20. Команда S A V E

Функция: сохраняет программу на СМП

Формат: SAVE "<спецификация файла>"[,A]

Эта команда сохраняет Бейсик-программу на СМП. Для этого указывается имя устройства, имя и тип файла. Если опущено имя устройства, то им считается рабочий СМП. Если отсутствует параметр A и тип файла, то записывается тип "BAC".

Если параметр A задан и отсутствует тип файла, то записывается тип "BAS".

Задание параметра A определяет сохранение программы в коде ASCII. Отсутствие параметра означает, что при записи используется внутренний формат. Он позволяет сократить обьем памяти, занимаемый программой на СМП. Этот обьем можно рассчитать при помощи команды MEM по формуле:

<обьем программы + 1 > / 512 = целая часть + остаток

если остаток равен 0, то число блоков, занимаемых программой будет равно целой части;
если остаток не равен 0, то число блоков равно целой части + 1.

Форматы команд SAVE и LOAD (см. п.5.1.13.) для данного файла должны совпадать.

Пример1:

```
SAVE    "ADDR"
( записывается файл ADDR.BAC на рабочий СМП во
  внутреннем формате)
```

Пример2:

```
SAVE    "SM1:PROG.ABC",A
( записывается файл PROG.ABC на СМП1 в коде
  ASCII )
```


5.1.21. Команда S Y S T E M

Функция: передает управление системному загрузчику.
Формат: SYSTEM.

Команда SYSTEM позволяет пользователю выйти в системный загрузчик без выключения машины.

5.1.22. Команда T I M E

Функция: устанавливает или сообщает текущее время
Формат: TIME [<чч>:<мм>:<сс>]

<чч>-часы
<мм>-минуты
<сс>-секунды

TIME с параметрами позволяет установить указанное время в таймере МК90.

TIME без параметров выдает текущее время. При полном отключении питания таймер МК90 останавливается.

Например: TIME 15:20:30
задано время 15 ч 20 мин 30 сек

5.1.23. Команда U S R

Функция : резервирует сегмент памяти для пользователя.
Формат : USR [<длина сегмента>]

Команда USR используется для резервирования сегмента памяти указанной длины из свободного пространства BASIC-интерпретатора и возвращает адрес пользователю.

Команда USR без параметров отменяет выделенный сегмент. Параметр <длина сегмента> указывает длину в байтах требуемого сегмента в восмеричной системе счисления. Команда USR печатает на ЖКИ адрес начала выделенного сегмента в восьмеричной системе счисления. Выделенная область памяти может использоваться для составления программ в машинных кодах, которыеможно набирать с помощью команды PATCH, сохранять командой BSAVE, загружать командой BLOAD и использовать в команде CALL.

При использовании данного сегмента контроль за переполнением должен осуществлять сам ползователь.

Если пользователю недостаточно первоначально выделенного сегмента, он может отменить данный сегмент и отвести сегмент большей длины.

Необходимо помнить, что резервирование пользовательского сегмента памяти приводит к сокращению памяти, отведенной для программ на бейсике в чем можно убедиться с помощью команды MEM.

5.2. Команды программного режима

Чаще всего программы на языке Бейсик записываются в программном режиме, где каждая строка начинается с номера, указывающего на последовательность выполнения. В программном режиме команды называют операторами.В этом режиме выполнение операторов задерживается до подачи специальной команды (например, RUN или GOTO).

Примеры программ приведены в приложении 4.

Программа, загруженная из СМП или набранная пользователем на клавиатуре, удаляется при выключении калькулятора либо в случае загрузки из СМП другой программы. При вводе строки с уже имеющимся номером последняя записывается на место строки с этим номером. В одной строке можно записать несколько операторов, которые разделяются двоеточием и выполняются последовательно.

5.2.1. Оператор C A L L

Функция: передает управление программе в машинных кодах, находящейся в памяти по абсолютному адресу.

Формат: CALL <адрес программы>[,<имя>]
<адрес программы> представляет собой выражение и задается в 10 с/с.
<имя>- имя переменной (или массива).
CALL #<адрес программы>[,<имя>]
<адрес программы> задается в 8 с/с.
<имя>- имя переменной (или массива).

Начальный адрес переменной (или первого элемента массива) передается в третьем регистре общего назначения. Значение переменной (или массива) должно быть определено до использования оператора CALL.

Переменная (или массив) представляются числами с плавающей запятой.

Размещение переменной (или массива) в памяти:

байт1	байт0	
имя2	имя1	имя1,имя2- имя переменной, массива
X	Y	X- размерность по X Y- размерность по Y
младшие разряды мантиссы		первый элемент массива или переменная
старшие разряды мантиссы		
порядок степени 2		
. . .		
младшие разряды мантиссы		последний элемент массива
старшие разряды мантиссы		
порядок степени 2		

Последней командой программы пользователя должна быть команда RTS PC (используется система команд ЭВМ "Электроника 60"), по которой передается управление в Бейсик. Содержимое используемых в программе регистров общего назначения и ячеек памяти перед командой RTS PC должно быть таким же, как и в момент передачи управления программе пользователя.

Использование команды USR <длина сегмента> дает возможность получить <адрес программы> в 8 с/с. Командой PATCH нужно ввести программу в машинных кодах в память.

В распоряжении пользователя имеются следующие программы Бейсика:

ATOF - упаковка десятичного числа в форму с плавающей

запятой;
ATOI - упаковка десятичного числа;
ITOA - распаковка десятичного числа со знаком
IMUL - умножение целых чисел;
ADDF - сложение чисел с плавающей запятой;
SUBF - вычитание чисел с плавающей запятой;
NEGF - изменение знака числа с плавающей запятой;
DIV - деление чисел с плавающей запятой;
MULF - умножение чисел с плавающей запятой;
CMPF - сравнение чисел с плавающей запятой;
FIX - преобразование числа с плавающей запятой в целое число;
FLT - преобразование целого числа в число с плавающей запятой.

Информация по использованию функций Бэйсика

Имя	Вызов TRAP	Ввод	Вывод	Исп. рег-ры
ATOF	104406	Указатель строки в R1	Число в форме с плавающей запятой (3 слова) записывается по адресу R0; R1 указывает на первый недопустимый знак	Все
ATOI	104410	Указатель строки в R1	Число в R0; R1 указывает на первый недопустимый знак	Все
ITOA	104412	Число в R1	Результат по адресу R0, где R0 указан при вводе	Все
IMUL	104416	Числа в R1 и R0	Старшие разряды в R0, младшие разряды в R1	Все
ADDF	104420	Указатели чисел в R0 и R1	Результат по адресу R0, где R0 указан при вводе	Все
SUBF	104422	Указатели чисел в R0 и R1	Результат по адресу R0, где R0 указан при вводе	Все
NEGF	104424	Указатель числа в R1	Результат по адресу R0, где R0 указан при вводе	Все
DIVF	104426	Указатели чисел в R0 и R1	Результат по адресу R0, где R0 указан при вводе	Все
MULF	104430	Указатели чисел в R0 и R1	Результат по адресу R0, где R0 указан при вводе	Все
CMPF	104432	Указатели чисел в R0 и R1	Те же признаки, что и при выполнении CMP	Все
FIX	104440	Числа в R2, R3, R4	Результат в R0	R0-R4
FLT	104436	Число в R1	Результат по адресу R0, где R0 указан при вводе	R0-R4

Пример: Определить <адрес программы>
USR 1000
036476

Ввести программу, используя команду PATCH.
PATCH

старое содержимое	программа в кодах	программа на ассемблере	
*36476/xxxxxx	010546	MOV	R5,-(SP)
*36500/xxxxxx	010446	MOV	R4,-(SP)
*36502/xxxxxx	010346	MOV	R3,-(SP)
*36504/xxxxxx	010246	MOV	R2,-(SP)
*36506/xxxxxx	010146	MOV	R1,-(SP)
*36510/xxxxxx	010046	MOV	R0,-(SP)
*36512/xxxxxx	005263	INC	4(R3)
*36514/xxxxxx	000004		
*36516/xxxxxx	012600	MOV	(SP)+,R0
*36520/xxxxxx	012601	MOV	(SP)+,R1
*36522/xxxxxx	012602	MOV	(SP)+,R2
*36524/xxxxxx	012603	MOV	(SP)+,R3
*36526/xxxxxx	012604	MOV	(SP)+,R4
*36530/xxxxxx	012605	MOV	(SP)+,R5
*36532/xxxxxx	000207	RTS	PC

*E

```
10  A=2
20  PRINT A
30  CALL #36476,A
40  PRINT A
50  END

RUN
 2
 4
ОСТ В СТРОКЕ 50
```


5.2.2. Оператор C L O S E

Функция: закрывает файл на внешнем носителе
Формат: CLOSE #<номер файла>

<номер файла>- номер канала, указанный в операторе OPEN для закрываемого файла.

Осуществляется закрытие файла и освобождение его номера для повторного использования.

5.2.3. Оператор C L S

Функция: очищает экран ЖКИ и переводит курсор в верхнюю левую позицию.

Формат: CLS

Используется для очистки экрана ЖКИ по мере необходимости.

После выполнения оператора курсор переходит в позицию (0,0), при этом изображение маркера отсутствует.

5.2.4. Оператор D E F F N

Функция: определение функций пользователя

Формат : DEF FN <буква>(<переменная>)=(выражение)

В некоторых программах возникает необходимость вычисления одного и того же математического выражения, часто с различными данными. В Бейсике имеется возможность определять функции или выражения и вызывать их так же, как и математические функции, т.е. синус, косинус, квадратный корень и т.д.

Определяемая пользователем функция идентифицируется именем, состоящим из трех букв, и аргументом в скобках. Первые две буквы имени функции - FN; третья буква может быть любой буквой латинского алфавита. Таким образом можно определить до 26 функций.

Такая функция должна быть определена прежде, чем она будет использована. Определяется она с помощью оператора определения функции DEF:

DEF FNa (параметр) = выражение,

где a - буква латинского алфавита.

Например, посредством оператора

```
10 DEF FNX (S)=S-2+4
```

оператор

```
20 LET R=FNX (4)
```

будет вычисляться как

```
LET R=4-2+4.
```

а оператор

```
20 LET R=FNX(2)
```

будет вычисляться как

```
LET R=2-2+4.
```

Параметры, указанные в определяемой пользователем функции, являются формальными параметрами. Параметры, указанные при вызове данной функции, являются фактическими параметрами.

В нашем примере S - формальный параметр, 4 и 2 - фактические параметры.

Аргумент в скобках может быть любым допустимым выражением; значение выражения подставляется вместо аргумента функции.

В следующем примере функция FNZ в строке 10 будет возводить в квадрат число, подставляемое вместо аргумента функции в строке 30 выражение 2+A дает 4; затем 4 возводится в квадрат и печатается значение функции FNZ(x).

Пример:

```
10 DEF FNZ(X)=X^2
20 LET A=2
30 PRINT FNZ(2+A)
Готов
RUN
16
Ост в строке 30
Готов
```

Другой пример:

```
10 DEF FNA(Z)=Z+A+B
20 DATA 2,4
30 READ A,B
40 LET F=FNA (9)+1
50 PRINT F
Готов
RUN
16
Ост в строке 50
Готов
```

Функция может использоваться рекурсивно,как показано в следующем примере. Выражение в строке 30 вычисляется как (2+(2*4)) прежде чем оно будет возведено в квадрат.

Пример:

```
10 DEF FNA(X)=X^2
20 LET A=2
30 PRINT FNA (2+A*FNA(2))
Готов
RUN
100
Ост в строке 30
Готов
```

Если одна и та же функция определяется несколько раз,используется первое определение, а последующие определения игнорируются.

Пример:

```
10 DEF FNX(X)=X^2
20 DEF FNX(X)=X+X
30 PRINT FNX(6)
40 END
Готов
RUN
36
Готов
```

Формальный параметр функции может не использоваться в выражении функции.В следующем примере подстановка фактического параметра вместо формального не оказывает никакого влияния на значение функции.

Пример:

```
10 DEF FNA (X)=4+2
20 LET R=FNA (10)+1
30 PRINT R
Готов
7
Ост в строке 30
Готов
```

5.2.5. Оператор D I M

Функция: объявляет массив.

Формат: DIM <идентификатор массива1>(< индекс1>[,<индекс2>])
[,< идентификатор массива2> (< индекс1)[,<индекс2>])...]

DIM резервирует место в памяти для массива данных.
Максимальное значение списка индексов - 255.
Идентификатором массива может быть любая переменная.
Максимальное значение размерности - 2, то есть допуска-
ется работа с одномерными и двумерными массивами. В опера-
торе должны содержаться максимально возможные индексы:

DIM A1 (4,4), B(10)

так как индексы в Бейсике начинаются с 0,приведенный выше оператор будет резервировать место для массива A1, способного вместить 25 элементов (5*5) и массива B - 11 элементов. Задавать размерность ранее использованной переменной недопустимо.

Например:

```
10 LET A3=C-C+B/3
   .
   .
40 DIM A3 (2,5)
```

вызовет сообщение об ошибке.

Применение.

Введем следующую программу:

```
10 DIM A(19)
20 FOR I=0 to 19
30 LET A(I)=0
40 NEXT I
60 FOR I=0 TO 4
70 INPUT A(I)
85 NEXT I
```

Выполним ее:

```
RUN    вк
   ? 5
   ? 6        ввод A(I)
   ? 7
   ? 8
   ? 9

    останов программы и переход в
Готов        режим команд
```


Посмотрим результат выполнения программы:

```
PRINT A
  5
PRINT A(1)
  6
PRINT A(2)
  7
PRINT A(3)
  8
PRINT A(4)
  9
PRINT A(5)
  0
PRINT A(6)
  0
PRINT A(29)

ошибка 6  стр  0
Готов
```

После ввода пяти констант выполнение программы останавливается. Затем используется оператор PRINT в непосредственном режиме для проверки начальных элементов массива. Попытка напечатать содержимое ячейки массива, находящейся за пределами, установленными оператором DIM, вызывает сообщение об ошибке.

Ниже приводится продолжение рассмотренной программы, демонстрирующей автоматический вывод.

```
 10 DIM A(19)
 20 FOR I=0 TO 19
 30 LET A(I)=0
 40 NEXT I
 60 FOR I=0 TO 4
 70 INPUT A(I)
 90 NEXT I
 95 REM - строки 100-130 печатают список последовательно
100 FOR I=0 TO 6
110 PRINT "A("I")", A(I)
120 NEXT I
130 STOP
135 REM - строки 140-160 печатают часть списка
136 REM - в обратном порядке
140 FOR I=6 TO 0 STEP-1
145 PRINT "A ("I")", A(I)
160 NEXT I
Готов
 RUN
 ? 1
 ? 2
```

```
? 3
? 4
? 5
A(0)     1
A(1)     2
A(2)     3
A(3)     4
A(4)     5
A(5)     0
A(6)     0
Ост в строке 130
Готов
Goto 140
A(6)  0
A(5)  0
A(4)  5
A(3)  4
A(2)  3
A(1)  2
A(0)  1
Ост в строке 160
Готов
```


5.2.6. Оператор D I S

Функция: восстанавливает служебную строку

Формат: DIS

Оператор DIS используется для восстановления служебной строки после использования графических команд, которые размещают информацию на месте, выделенном под служебную строку.

5.2.7. Оператор D R A W

Функция: выполняет графичеокие команды для ЖКИ.
Формат: DRAW <команда><список параметров>

1) Команда O

Функция: задает текущую точку экрана.

Формат: DRAW O<координата X>,<координата Y>
Координаты точки X,Y ограничены размерами экрана:
0 =< X <= 119, 0 =< Y <= 63.

2) Команда H

Функция: высвечивает точку в заданной позиции экрана.
Формат : DRAW H<координата X>,<координата Y>

Координаты точки X,Y ограничены размерами экрана:
0 =< X <= 119 , 0 =< Y <= 63

3) Команда D

Функция: вычерчивает отрезки прямых между последовательностью заданных точек

Формат: DRAW D<начальная координата X>,<начальная координата Y>,<координата X>,<координата Y>[,...]

Используется для высвечивания на экране точки,прямой линии,ломаной линии,заданных абсолютными координатами X,Y.

Координаты точек X,Y ограничены размерами экрана:

0 =< X <= 119, 0 =< Y <= 63

Если соседние пары координат X,Y одинаковы, то команда вычерчивает в данной позиции точку.

```
Пример:     5   CLS
           10 DRAW     D50,30,50,30
           20 DRAW  D70,5,70,5
           25   DIS
```


4) Команда E

Функция: гасит точку(линию)в заданном положении.

Формат: DRAW E<начальная координата X>,<начальная координата Y>,<координата X>,<координата Y>
Используются для гашения на экране точки,линии, заданных абсолютными координатами X,Y.

Координаты точек X,Y ограничены размерами экрана:

0 =< X <= 119, 0 =< Y <= 63.
Если соседние пары координат X,Y одинаковы,то команда гасит в данной позиции точку.

```
Пример:     5   CLS
           10   FOR   I=1   TO 3
           20   DRAW  H10,I
           25   REM-задержка
           30   FOR   J=1 TO  20: NEXT J
           40   DRAW  E10,I,10,I
           50   NEXT  I
```

5) Команда I

Функция: вычерчивает прямую линию,координаты которой заданы относительно.

Формат: DRAW I<приращение по X>,<приращение по Y>

Используется для высвечивания отрезков прямых,координаты конечных точек которых заданы приращениями относительно текущей точки экрана.

```
Пример:     5   CLS
           10   DRAW   020,32
           20   DRAW I0,0,20,10,-5,-30,-10,0
           30   REM-задержка
           40   FOR J=1 TO 200: NEXT J
           50   REM-установить служебную строку, маркер
           60   DIS
```

6) Команда A

Функция: вычерчивает прямоугольник,заданный координатами его диагонали.

Формат: DRAW A<начальная координата X>,<начальная координата Y>,<диагональная координата X>, <диагональная координата Y>

Используется для вычерчивания прямоугольника,координаты которого задаются оледующим образом:

```
     X1,Y1  X0,Y0          X0,Y0     X1,Y1
  ----------  ----------   ----------  ----------
  I         I I         I  I        I  I         I
  ----------  ----------   ----------  ----------
  X0,Y0          X1,Y1     X1,Y1          X0,Y0
```

Координаты точек X,Y ограничены размерами экрана:

0 = < X < = 119, 0 = < Y < = 63

```
Пример:  5   CLS
 10    DRAW  060,32
 20    FOR   I=30   TO  5  STEP  -5
 30    DRAW  A60-I,32+I,60+I,3"-I
 40    NEXT     I
```

Результат выполнения программы приведен на рис.6.4.

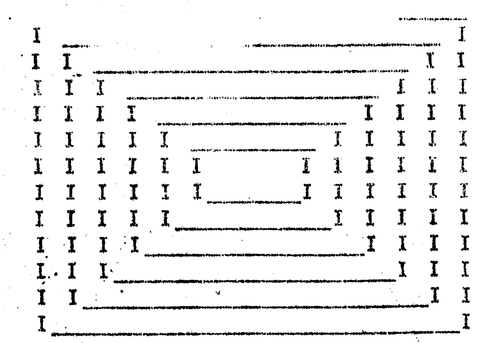


рис.6.4.

7) Команда X

Функция: вычерчивает оси координат.

Формат: DRAW X<направление оси координат>,<размер деления оси>,<количество делений>

Используется для высвечивания осей координат для вывода линейных графиков и гистограмм.

Параметр направления оси координат принимает значения

0,1,2, или 3.

0 - вычерчивается ось Y из начала координат вверх (+Y)
1 - вычерчивается ось X из начала координат вправо(+X)
2 - вычерчивается ось Y из начала координат вниз (-Y)
3 - вычерчивается ось X из начала координат влево (-X)

Следующие параметры - целые положительные числа,задают расстояние между метками шкалы в точках и количество меток шкалы соответственно.

```
Пример:   5   CLS
  10    DRAW  060,32
  20    FOR   I=0  TO  3
  30    DRAW  XI,3,3
  40    NEXT  I
```

Графически пример представлен на рис.6.5.

рис.6.5.

8) Команда G

Функция: высвечивание вертикальных или горизонтальных полос заданной ширины.

Формат: DRAW G<направление штриховки>,<протяженность по оси X>,<протяженность по оси Y>[,<расстояние между линиями>]

Используется для штриховки в заданном направлении прямоугольной области.Начальной точкой для штриховки служит текущая точка.

Параметр направления штриховки принимает значения 0,1,2

0 - штриховка не выполняется
1 - горизонтальные полосы
2 - вертикальные полосы

Параметр <расстояния между линиями> может быть опущен.

```
Пример:   5  CLS
         10  DRAW  00,0
         20  FOR   I=0 TO 2
         30  DRAW  AI*30,20,I*30+20,0
         35  LOCATE I*30,0
```

```
        40  DRAW  GI,20,20,2
        50  NEXT  I
```

На экране рисуются 4 квадрата. Первый не заштрихован, второй штрихуется 9-ю горизонтальными линиями, а третий - 9-ю вертикальными.

9) Команда C

Функция: вычерчивает окружность по заданным координатам центра и радиусу.
Формат : DRAW C<координата X центра >,<координата Y центра >,<радиус>

Используется для вычерчивания окружности заданного радиуса.Центр окружности задан абсолютными координатами (X,Y), ограниченными размерами экрана:
0 =< X <= 119, 0 =< Y <= 63 и <X> не равно <Y>.

```
Пример: 10  CLS
        20  FOR I=1 TO 3
        30  DRAW C60,32,I*10
        40  NEXT I
```

10) Команда M

Функция: высвечивает маску, заданную шестнадцатеричным числом.
Формат: DRAW M<маска>

Используется для заполнения прямоугольных областей заданной <маской>. Числа из заданной маски выбираются попарно и формируется 8 точек экрана по горизонтали. Заполнение областей осуществляется сверху вниз от текущего положения маркера или позиции, заданной командой LOCATE.
Позволяет выводить нерегулярные фигуры с произвольной конфигурацией.

Высвечивание точек выполняется следующим образом:

маска	двоичный эквивалент	
F0F0	11110000 11110000	
	****---- ****----	высвеченные точки
	....	(*-точка высвечена,)
		(- -точка не высвечена)
AAAA	10101010 10101010	
	--*-*- *-*-*-*-	

Если текущее положение маркера или позиции равно точке с координатами (X,Y), то при работе оператора DRAW/M для каждой из пар шестнадцатиричных чисел происходит автоматическое изменение координаты Y,т.е. Y=Y+1. При Y=64, автоматически полагается Y=0,а X=X+8

При X>119 и Y>63 маски не отображаются.

Пример:

```
        10  CLS
        20  FOR I=1 TO 120
        30  DRAW MF0F0F0F00F0F0F0F
        40  NEXT I
```

```
* * * * . . . .
* * * * . . . .
* * * * . . . .
* * * * . . . .
. . . . * * * *
. . . . * * * *
. . . . * * * *
. . . . * * * *
```

5.2.8. Оператор E N D

Функция: последний оператор программы
Формат: END

Оператор END следует включать в программы, предназначенные для неоднократного использования. Он должен помещаться в конце программы, т.е. в строке, являющейся логическим концом программы.

5.2.9. Оператор F O R - TO - S T E P / N E X T

Функция: FOR-TO-STEP используется для обозначения начальной точки цикла программы.
Формат: FOR <переменная>=e1 TO e2 [STEP e3],
где e1, e2, e3-выражения
Функция: NEXT используется для обозначения конечной точки цикла программы
Формат: NEXT<переменная>

Все операторы, стоящие между FOR и соответствующим ему оператором NEXT, циклически выполняются в соответствии с условиями, заданными в операторе FOR. Переменная, следующая за оператором FOR, рассматривается как управляющая переменная. Та же переменная должна присутствовать в операторе NEXT, определяющем конец цикла.

Минимальное и максимальное значения, представленные выражениями слева и справа от слова "TO" в операторе FOR, являются областью действия управляющей переменной.
Выражение, стоящее после слова STEP, указывает на какую величину изменится управляющая переменная после каждого прохода цикла. Если e3 не задано, шаг цикла равен 1.

пример:

```
10 LET A=5
20 FOR B=1 TO 5
30 PRINT A+B
40 NEXT B
Готов
RUN
 6
 7
 8
 9
 10
Ост в строке 40
Готов
```

В строке 20 переменной "B" присваиваются значения 1,2,3,4, 5.Так как она не может иметь все эти значения одновременно, создается цикл, начинающийся с оператора FOR в строке 20 и заканчивающийся оператором NEXT в строке 40.Операторы внутри цикла выполняются пять раз,причем каждый раз переменная "B" получает новое значение с шагом единица и печатается значение A+B. Оператор NEXT вызывает переход на строку 20 до тех пор, пока "B" не достигнет своего конечного значения 5.

Каждый оператор FOR внутри программы должен сопровождаться оператором NEXT; NEXT не может использоваться без предшествующего оператора FOR.

5.2.10. Оператор G E T

Функция: чтение значения переменной с внешнего устройства
Формат: GET #<номер>,<имя>

<номер> - номер канала, указанный в операторе OPEN и идентифицирующий данный файл. Для чтения переменной, файл на СМП должен быть уже создан и открыт оператором OPEN как вводной файл.
<имя> - имя переменной или элемента массива куда будет считана запись.

Осуществляется считывание очередной записи из файла на внешнем носителе и присвоение ее в качестве значения заданной переменной или элемента массива.

Пример:

```
10 DEV "SM1:"
20 DIM A(5)
30 OPEN #1,I,"INP"
40 FOR I=0 TO 4
50 GET #1,A(I)
60 PRINT A(I);
70 NEXT I
80 PRINT
90 GET #1,B
100 PRINT B
```

Файл INP.BAS на СМП1 открыт для ввода с номером канала 1. Предположим, в нем содержится следующая последовательность записей: 0,1,4,9,16,64. В результате выполнения прграммы, элементам массива A (I) присваиваются значения:

A (0)=0
A (1)=1
A (2)=4
A (3)=9
A (4)=16,

а переменной B присвоится значение 64 .

На экране будет напечатано:

```
0      1      4      9      16
64
```


5.2.11. Операторы: G O S U B , R E T U R N

Функция: позволяет организовать переход к подпрограмме
Формат: GOSUB <номер строки>

Номер строки, которой передается управление может быть задан числом или выражением. Выражение до передачи управления преобразуется в целое положительное число (см. функции ABS,INT).

Функция: последний оператор подпрограммы. Передает управление оператору, следующему за соответствующим оператором GOSUB.
Формат: RETURN

Часто в программе необходимо записать группу операторов (подпрограмму) только один раз, а затем неоднократно обращаться к подпрограмме из различных точек программы. Оператор GOSUB используется для перехода к первому оператору программы. Заканчивается подпрограмма оператором RETURN.

Оператор RETURN должен использоваться с GOSUB. Однако, одного оператора RETURN достаточно для случая, когда несколько GOSUB используются для перехода к одной программе.

например:

```
 5 PRINT " X", " X^2", " X^3"
10 FOR A=1 TO 3
15 LET X=A
20 GOSUB 50
25 NEXT A
28 FOR A=1 TO 3
30 LET X=A^2
40 GOSUB 50
45 NEXT A
48 STOP
50 FOR J=1 TO 2
51 PRINT X^J,
52 NEXT J
53 PRINT X^3
54 RETURN
55 END
Готов
RUN
X     X^2     X^3

1       1       1
2       4       8
3       9      27
1       1       1
4      16      64
9      81     729
Ост. в  строке 48
Готов
```

Вход в подпрограмму (строки 50-54) осуществляется операторами GOSUB в строках 20 и 40, а выход из нее - оператором RETURN в строке 54.

5.2.12. Оператор G O T O

Функция: передает управление строке N и продолжает выполнение программы с этой строки

Формат: GOTO <номер строки>

Оператор GOTO осуществляет безусловный переход к строке с указанным номером. Выполнение программы продолжается последовательно, начиная с оператора, к которому был произведен переход. Номер строки может быть задан числом или выражением. Выражение до передачи управления преобразуется в целое положительное число (см. функции ABS, INT).

Пример:

```
10 DATA 1,2,3,4,5 (см. п.6.3.2.23.-описание операторов DATA и READ)
15 READ X
20 PRINT X+1
40 GOTO 15
Готов
RUN
 2
 3
 4
 5
 6
Ошибка 20 строке 15
Готов
```

Операторы 15,20 и 40 представляют цикл в приведенном примере. Эти три оператора непрерывно выполняются с новым значением X при каждом выполнении строки, пока не будет использовано последнее значение константы оператора DATA.

Выполнение программы заканчивается сообщением об ошибке, показывающим, что использованы все значения констант.

В следующем примере оператор GOTO осуществляет переход к оператору INPUT так, что программа зацикливается.

```
10 INPUT A      ( см.п.6.3.2.14 - описание оператора
20 PRINT A        INPUT )
30 LET A=A^A
40 PRINT A
50 PRINT
60 GOTO 10
Готов
RUN
? 2
  2
  4

? 3
  3
  27

? 4
  4
  256
```

```
? 5
  5
  3125
 ---- ---
I CY I P I
 ---- ---
Готов
```

При записи в строке нескольких операторов, разделяемых символом ":", оператор GOTO должен быть последним оператором в строке.

5.2.13. Оператор I F/ T H E N-

Функция: обеспечивает условный переход к программе

Форматы: .IF<выражение> THEN <оператор>
IF<выражение> THEN <номер строки>
IF<выражение> GOTO <номер строки>

За словом THEN может следовать любой оператор, включая второй оператор IF. Когда за THEN следует номер строки, оператор IF-THEN действует также, как и IF-GOTO.

Примечание: буква "T" не должна использоваться как наименование переменной в операторе IF.

Когда Бейсик встречает оператор IF, он вычисляет выражения и сравнивает их в соответствии с требованиями логической операции согласно табл.6.1.

Таблица 6.1.

Логическая операция языка Бейсик.	Пример	Значение
=	a=в	а равно в
<	a<в	а меньше в
<=	a<=в	а меньше или равно в
>	a>в	а больше в
>=	a>=в	а больше или равно в
<>	a<>в	а не равно в

Если условие истинно, выполняется остальная часть оператора, начинающаяся с THEN или GOTO. Если условие ложно, выполняется следующий оператор после оператора IF.

Пример:

используется формат IF GOTO <номер строки>

```
10 FOR A=1 TO 10
```

```
 20 LET X=INT(A^2)
 25 IF X>20 GOTO 40
 30 PRINT X
 35 NEXT A
 40 PRINT "значение A=" A
 55 STOP
 Готов
 RUN
  1
  4
  9
 16
значение A=5
Ост в строке 55
 Готов
```

Если заменить строку 25 на IF X>9 GOTO 40, то цикл заканчивается, когда будет A=3.

```
 RUN
  1
  4
  9
 значение A=4
 Ост в строке 55
 Готов
```

В следующем примере в строках 40,50 используется формат IF-THEN<оператор>:

```
 10 REM-программа оценки успеваемости
 25 INPUT A,B,C,D
 30 LET X=(A+B+C+D)/4
 40 IF X>=70 THEN PRINT X; "удовлетворительная оценка"
 50 IF X<70 THEN PRINT"неуспевающий на курсе"
 55 GOTO 25
 Готов
 RUN
 ?55,67,78,89
 72.25 удовлетворительная оценка
 ? 55,44,32,21
 неуспевающий на курсе
  ____ ___
 ?I CУ,I P I
  ____ ___
 Готов
```

Оператор IF может записываться в любом месте в строке с несколькими операторами за исключением случая, когда за THEN следует оператор GOSUB или GOTO.В этом случае оператор IF должен быть последним оператором в строке. Оператор IF-THEN-GOSUB обеспечивает переход к подпрограмме и должен быть последним оператором в строке.

5.2.14. Оператор I N P U T

Функция: осуществляет ввод данных с клавиатуры во время выполнения программы.
Формат: INPUT V1[,V2,...,VN]

Бейсик во время выполнения оператора INPUT печатает знак вопроса и ждет,когда пользователь введет от единицы до N числовых значений.

При выполнении оператора INPUT числовые значения должны разделяться запятыми и заканчиваться нажатием ВК. Для каждого оператора INPUT печатается только один знак вопроса.

Если введено недостаточное количество данных,Бейсик печатает:

ош 121 стр NNN

если введено слишком много данных,Бейсик печатает:

ош 122 стр NNN.

где NNN - номер строки оператора INPUT.

В обоих случаях Бейсик печатает второй знак вопроса и ждет выполнения повторного ввода.Строка ввода должна быть перепечатана.Выход в командный режим осуществляется по нажатию CУ/P.

Оператор INPUT может записываться в любом месте программы, а также может включаться в строку с несколькими операторами.

пример:

```
 10 INPUT A,B,C
 20 GOTO 10
 RUN
 ?1
 ?2
 ?3
  ____ ___
 ?I CУ I P I
  ____ ___
```

5.2.15. Оператор L E T

Функция: присваивает значение выражения переменной
Формат: [LET] <переменная>=<выражение>

Следующий пример иллюстрирует использование оператора LET:

```
10 LET A=1
20 LET B=2
30 LET C=A+1
40 LET X=A+B+C+1
```

После выполнения этих операторов значение X будет равно 6. Оператор LET может записываться в любом месте программы и в строке с несколькими операторами. Оператор LET можно опускать.

Пример:

```
B=2
```

После выполнения значение B будет равно 2.

5.2.16. Оператор L O C A T E

Функция: задает позицию курсора.

Формат: LOCATE <координата X>,<координата Y>

Используется для изменения положения курсора. Координаты X,Y ограничены размерами экрана:

0 =< X <= 119, 0 =< Y <= 63

Пример:

```
1)  5 CLS
   10 LET X=1
   20 LET X=X+1
   40 PRINT   "X=";X
   50 GOTO    20
2)  5  CLS
   10 LET X=1
   20 LET X=X+1
   30 LOCATE 20,20
   40 PRINT "X=";X
   50 GOTO  20
```

В случае 1) печать результата начинается с текущей позиции курсора.

В случае 2), в связи с добавлением строки 30, печать результата начинается с позиции (20,20).

5.2.17. Оператор O P E N

Функция: разрешает доступ к файлу на внешнем устройстве

Формат: OPEN #<номер файла>, I/O, " < спецификация файла > "

<номер файла> - числовой идентификатор файла или набора данных. Определяет номер канала и файла, принимает значения 1 - 6.

Параметр I/O принимает одно из следующих значений:

I - файл предназначен для чтения (входной файл)

O - файл предназначен для записи (выходной файл)

Параметр " < спецификация файла > " имеет следующий формат:

<[SMK:]< имя файла >[<.тип>], где

SM - тип устройства ввода/вывода (СМП)

K - номер СМП. Может принимать значения 0, 1.

Если имя и номер устройства опущены, то за устройство ввода/вывода принимается рабочий СМП (см п.6.3.1.7). При отсутствии параметра <.тип>, на СМП записывается файл с типом ".BAS".

Осуществляется присваивание заданного номера поименованному файлу и разрешение доступа к данным, хранящимся в этом файле. Одна запись файла представляет собой целое число записанное в формате ASCII. Просмотреть содержимое файла можно с помощью оператора LIST (см. п.6.3.1.12.). В зависимости от значения параметра I/O программа может записывать данные с начала нового файла, добавлять их в конец уже существующего (выходной файл) или считывать данные из файла (входной файл).

Пример:

```
10 DEV "SM1:"
20 OPEN #1,I,"INP"
```

Файл INP.BAS на СМП1 открыт для ввода с номером канала 1.

Одновременно для ввода/вывода может быть открыто до 6 файлов.

5.2.18. Оператор P A I N T

Функция: заполняет заданную область экрана указанной маской

Формат: PAINT <номер столбца>, <номер строки>, <номер маски>

<номер столбца>, <номер строки> - координаты внутренней точки заполняемой области, <номер маски> - числовое значение. 0 <= <номер маски> =< 48. Соответствие номера маски и рисунка приведено в Приложении 5.

Закрашивание продолжается до тех пор, пока не встретятся точки, которые являются:

1) границами заданной области (заданная точка является внутренней);

2) границами экрана ЖКИ (заданная точка является внешней).

Если какой-либо параметр не задан, то на экран выдается сообщение о неправильном задании параметров.

Если задана точка, лежащая на границе фигуры, то никаких действий на экране не происходит.

Пример.

```
10 CLS
20 DRAW C60,32,30
40 PAINT 74,40,3
```

На экране рисуется окружность и закрашивает я маской 3.

5.1.19. Оператор P L A Y

Функция: генерирует тон заданной частоты и длительности.
Формат: PLAY <тон>,<длительность>.
Параметр <тон> задает номер тона от 0 до 40
Параметр <длительность> определяет продолжительность звучания генерируемой тона и задается числом от 1 до 32767. Длительность целой ноты 3-4 сек. Тон равный "1" соответствует "ФА" большой октавы.

Пример: программа воспроизведения мелодии

"Подмосковные вечера"

```
10 FOR J=0 TO 2
20 READ K
30 FOR I=1 TO K
40 READ A,B,C
50 PLAY A,B*5
60 WAIT C
70 NEXT I
80 RESTORE
90 NEXT J
100 END
110 DATA 54
120 DATA 20,2,1,23,2,1,27,2,1,23,2,1,25,4,1,23,2,1
130 DATA 22,2,1,27,4,1,25,4,1,20,8,1 23,2,1,27,2,1
140 DATA 30,2,1,30,2,1,32,4,1,30,2,1,28,2,1,27,8,1
150 DATA 29,4,1,31,4,1,34,2,1,32,2,1,27,6,1,22,4,1
160 DATA 20,2,1,27,3,1,25,1,1,28,6,1,30,2,1,28,2,1
170 DATA 27,4,1,25,2,1,23,2,1,27,4,1,25,4,1,32,8,1
180 DATA 29,4,1,31,4,1,34,2,1,32,2,1,27,6,1,22,4,1
190 DATA 20,2,1,27,3,1,25,1,1,28,8,1,30,2,1,28,2,1
200 DATA 27,4,1,25,2,1,23,2,1,27,4,1,25,4,1,20,12,1
```


5.2.20. Оператор P R I N T

Функция: осуществляется вывод данных на ЖКИ
Формат: PRINT [<функция>][<список>]

Список может содержать выражение и (или) строку текста. Элементы списка разделяются с помощью "," или ";", которые задают формат печати.
Оператор PRINT без списка используется с целью вывода пустой строки.
PRINT может использоваться для вычислений: производится вычисление выражения, содержащегося в списке, и вывод на печать его значения. Полученное значение не сохраняется для последующего использования.

```
10 LET A=1
15 LET B=5
20 PRINT A+B
Готов
RUN
 6
ост в строке 20
```

Значения выражений, разделенных в операторе PRINT символом ",", печатаются через четыре позиции справа от предыдущего. Если при печати требуется более компактное расположение значений, используется символ ";" .В этом случае каждое значение печатается через две позиции справа от предыдущего, при условии, что эта строка еще не закончена.
Ниже приведен пример задания форматов печати.

```
     10  DATA   1,2,3
     20  READ A,B,C
     30  PRINT A;B;C;
     40  PRINT A;B;C
     50  PRINT A,B,C
     60  PRINT A,B,C
     Готов
     RUN
1  2  3  1  2  3
1    2    3
1    2    3
```

Для печати сообщений, комментариев или любой строки знаков оператор PRINT имеет функции, приведенные в табл.5.2.

Таблица 5.2.

S	изменение размеров символов и знаков	
Q	задание ориентации символов и знаков	
Y	задание направления печати	
Z	задание расстояний между символами и между строками	
N	вывод информации в негативе	
P	вывод информации в позитиве	

Список функций располагается за оператором PRINT и заключается в угловые скобки <> , между собой функции разделяются символом ":". Текст, который должен быть напечатан заключается в кавычки.

Функции оператора PRINT имеют следующие форматы:

1) Функция: S
Формат: S<размер символа по X, размер символа по Y
Параметр размера символа означает во сколько раз размеры данного символа превосходят стандартные размеры (S#7).
По умолчанию параметр равен "1".
Например, символ "M" можно печатать с использованием размеров от 1 до 3 следующим образом:

```
10 CLS
20 DRAW 060,32
30 FOR I=1 TO 3
40 PRINT <SI,I>"M";
50 NEXT I
```

2) Функция: Q
Формат: Q<тип ориентации>
Параметр типа ориентации принимает значения 0,1,2,3.
Приведем пример печати буквы "F" в разных ориентациях:

```
10 FOR I=0 TO 3
20 PRINT <QI>"F":PRINT I
30 NEXT I
```

3) Функция: Y
Формат: Y<направление печати>
Параметр направления печати принимает значения 0,1,2,3.

0 - печать символов слева направо

1 - печать символов сверху вниз
2 - печать символов справа налево
3 - печать символов снизу вверх

Функцию Y рекомендуется использовать с LOCATE ,т.к.символы, выходящие за пределы экрана ,не отображаются.
Пример:

```
PRINT <Y1>"ABW"
```

4) Функция: Z
Формат: Z <расстояние между символами, расстояние между строками>

Вывод информации с заданным расстоянием между символами и строками. Действует до конца оператора PRINT.

5) Функция: N
Формат: N

Вывод информации в негативе. Действует до функции <P> или до конца оператора PRINT.

6) Функция: P
Формат: P

Вывод информации в позитиве. Действует до функции <N> или до конца оператора PRINT.

5.2.21. Оператор P U T

Функция: запись значения переменной на внешнее устройство
Формат: PUT #<номер>,<имя>

<номер>- номер канала, указанный в операторе OPEN и идентифицирующий файл. Файл должен быть открыт оператором OPEN как вводной файл.
<имя> - имя выводимой переменной, элемент массива или выражение.

Осуществляется занесение значения переменной в файл на внешнем носителе. Запись значения происходит за последней записью уже существующего файла или в начало нового файла. Просмотреть содержимое файла можно с помощью оператора LIST (см.п.6.3.1.12).

Пример:

```
10 DEV "SMI:"
20 OPEN #1,0,"INP"
30 DIM A (5)
40 FOR I=0 TO 4
50 A(I)=I^2
60 PUT #1,A(I)
70 NEXT I
80 PUT #1,I^3
90 CLOSE #1
```

Файл INP.BAS открыт на СМП1 для вывода с номером канала 1. В результате выполнения прграммы в файл записывается следующая информация:

0,1,4,9,16,64

5.2.22. Оператор R A N D O M I Z E

Функция: позволяет получать ряд случайных чисел в программе, используя функцию RND.
Формат: RANDOMIZE

При выполнении оператора RANDOMIZE функция RND выбирает начальное значение для получения случайного числа.

Пример:

```
10   REM - случайные числа с использованием
     RANDOMIZE
15   RANDOMIZE
25   PRINT "случайные числа:"
30   FOR  I=1 TO  4
40   PRINT  RND (0)
50   NEXT  I
60   END


RUN
случайные числа:
.5157776
.5863953
.8763733
.9806824
Ост в строке 60
Готов

RUN
случайные числа:
.837677
.9583435
.2109681
.6407166
Готов

RUN
случайные числа:
.2499695
.7420959
.2028503
.5382385
```

При удалении оператора RANDOMIZE и изменении строки 25 PRINT "генерирование случайных чисел:"
Получаем следующее:

```
RUN
Генерирование случайных чисел:
.1002502
.9648132
.8866272
.6364441
```

```
Ост в строке 60
Готов
RUN

Генерирование случайных чисел:
.1002502
.9648132
.8866272
.6364441
Ост в строке 60
RUN

Генерирование случайных чисел:
.1002502
.9648132
.8866272
.6364441
Ост в строке 60
```


5.2.23. Операторы: R E A D , D A T A , R E S T O R E

Функция: переменным от V1 до VN включительно присваиваются значения соответствующих констант оператора DATA
Формат: READ V1[,V2,...,VN]

Функция: определение значений констант N1,...,NN для оператора READ
Формат: DATA N1,[N2,...,NN]

Функция: устанавливает указатель в начало строки констант DATA
Формат: RESTORE

Для присвоения значений одиночным переменным в предыдущих примерах использовался оператор LET; для большего числа переменных использовалось и большее количество операторов LET.

Однако в программах со многими переменными для их определения следует использовать READ и DATA.

Оператор DATA вводит в программу числовую константу или группу констант. Оператор READ последовательно связывает имена переменных со значением констант, которые задаются операторами DATA.

Операторы READ и DATA должны сопровождать друг друга в программах пользователей, но они не обязательно должны быть парными. Если в одном или более операторах READ содержится 9 переменных, то в одном или более операторах DATA должно содержаться по крайней мере девять констант (исключение составляет использование оператора RESTORE).

В следующем примере все константы задаются одним оператором DATA. Они используются операторами READ в разных строках.

```
10 DATA 1,5,3,7,9
20 READ A,B
30 LET X=A+B
40 PRINT A;B;X
50 READ U,O,R
60 LET X=X+U+O+R
70 PRINT X;U;O;R
Готов
RUN
1   5   6
25   3   7   9
Ост в строке 70
Готов
```

При выполнении программы Бейсик игнорирует оператор DATA до тех пор, пока не встретится оператор READ. Затем он осуществляет поиск оператора DATA и находит его в строке 10 т.е. в первой строке программы.

Бейсик последовательно берет значения констант и связывает их с переменными в операторе READ, которые также выби-

раются последовательно:переменной A присваивается значение 1 B-значение 5.Установив указатель на следующем элементе данных 3, он переходит на строку 30,т.е. к следующему невыполненному оператору.

В строке 50 встречается другой оператор READ,который содержит три новых переменных.Теперь Бейсик обращается к указателю для получения следующей неиспользованной константы. Переменным U,Q,R присваивается значение констант 3,7,9.

Оператор RESTORE дает возможность осуществить повторное использование конотант операторов DATA,начиная с операторов DATA с самым низким номером строки в программе.

```
Пример: 40  DATA  1,2
  50  READ  A,B
  60  PRINT A;B
  70  RESTORE
  80  READ  C,D
  90  PRINT C+D;C;D
  Готов
  RUN
  1  2
  3  1  2
  Ост в строке 90
  Готов
```

Если бы не было оператора RESTORE, в строке 80,появилось бы сообщение об ошибке,указывающее на отсутствие конотант для оператора READ.

```
Пример:   10 DATA  1,2
    20   READ  A,B
    30   PRINT A;B
    40   RESTORE
    50   DATA 3,4
    60   READ  C,D
    70   PRINT  C+D;C;D
    Готов
    RUN
    1  2
    3  1  2
```

в данном примере под действием оператора RESTORE второй оператор READ в строке 60 получает значение конотант первого оператора DATA в строке 10, а не второго, находящегося в строке 50.

```
Пример:  10   DATA   1,2
   20   READ   A,B
   30   PRINT  A;B
   40   DATA   3,4
   50   READ   C,D
   60   PRINT  C+D;C;D
   Готов
   RUN
   1  2
```


7 3 4

Оператор DATA нельзя включать в строку с несколькими операторами; он должен быть единственным на нумерованной строке.

Оператор READ может быть записан в любом месте в строке с несколькими операторами.

Оператор RESTORE может использоваться в любом месте программы;он может включаться в строку с несколькими операторами.

Оператор RESTORE не имеет действия в программах без операторов DATA и READ.

5.2.24. Оператор R E M

Функция: позволяет напечатать комментарии в программе .
Формат: REM - <комментарий>

Комментарии могут состоять из любых знаков
Операторы REM не оказывают действ я на выполнение программ, однако они занимают часть памяти.
При использовании в строке с несколькими операторами, REM должен записываться последним,т.к. Бейсик игнорирует все, что идет в строке после REM.
Оператор REM может быть заменен открывающейся кавычкой <'>.

5.2.25. Оператор S T O P

Функция: останов выполнения программы.Бейсик печатает:
ост в строке xxx
Готов
Формат: STOP

Оператор STOP останавливает выполнение программы, может помещаться в любом месте программы.
Оператор STOP помогает при отладке программ пользователя, он может помещаться в критических точках программы, выполнение программы останавливается в этих точках.При останове для просмотра значений конкретных переменных можно использовать оператор PRINT в непосредственном режиме.
Если в этой точке выполнение программы оказывается правильным, она может быть продолжена оператором CONT в непосредственном режиме,который обращается в этом случае к оператору,следующему за оператором STOP.

5.2.26. Оператор T R O F F

Функция: отменить режим трассировки программы.
Формат: TROFF.

Оператор TROFF используется для отмены режима трассировки.

Команда RUN не отменяет режим трассировки. Любая команда, удаляющая текущую программу из памяти (например, команда NEW или LOAD), отменяет и режим трассировки.

Вызов оператора TROFF в любом из режимов-программном или непосредственном-приводит к отмене режима трассировки последующих программ.

5.2.27. Оператор T R O N

Функция: установить режим трассировки программы.
Формат: TRON.

Оператор TRON приводит в действие трассировочную программу и помогает пользователю определить, где именно в программе возникает ошибка, облегчая процесс отладки больших и сложных программ.

Оператор TRON в процессе работы программы пользователя выводит номера ее строк, заключенных в квадратные скобки, в той последовательности, в которой они реально исполнялись. Цепочка этих номеров составляет след (или трассу) работы программы. Оператор TRON может работать как в программном так и в непосредственном режимах.

Для отмены режима трассировки используется оператор TROFF.

Команда RUN не отменяет этого режима, поэтому использование команды TRON в непосредственном режиме приведет к выполнению трассировки всех последующих программ до тех пор, пока не встретится оператор TROFF.

Любая команда, удаляющая текущую программу из памяти (например, команда NEW или LOAD), отменяет и режим трассировки.

Пример:

```
10  TRON
20  K=0
30  GOSUB 100
40  END
100 IF K>5 THEN GOTO 120
110 K=K+1: GOTO 100
120 RETURN
RUN
[10] [20] [30] [100] [11
0] [100] [110] [100] [11
0] [100] [110] [100] [11
0] [100] [120] [40]
ГОТОВ
_
```

5.2.28. Оператор W A I T

Функция: приостанавливает выполнение программы на заданную задержку.
Формат: WAIT <переменная>

Значение переменной - целое число в пределах от 1 до 32767.

Минимальная задержка равна 1/16 сек при значении переменной, равном 1.

Максимальная задержка равна 34 мин при значении переменной, равном 32767.

Пример:

```
5  REM  ПОЛУЧЕНИЕ ЗАДЕРЖКИ ДЛИТЕЛЬНОСТЬЮ 1 СЕК
10 WAIT 16
20 END
```

Для получения задержки длительностью N сек переменная =N*16, где 1/16<=N<=2047.

5.3. Встроенные функции

Бейсик содержит десять функций для выполнения вычислительных операций. Эти функции обозначаются именем, состоящим из трёх букв, за которым следует аргумент в круглых скобках. Функции могут использоваться в любом месте программы в качестве выражений или элементов выражений везде, где выражения являются допустимыми.

5.3.1. Функция A B S (X)

Абсолютная функция ABS вычисляет абсолютное значение любого аргумента.

Пример:

```
10 INPUT  X
20 LET X=ABS(X)
30 PRINT X
40 GOTO 10
RUN
?-35.7
 35.7
?2
 2
?105555567
 .1055556E 9
?-44.555566668899
 44.55557
  ---- ---
? ICy I P I
  ---- ---
Готов
```


5.3.2. Функция A T N (X)

Функция арктангенс вычисляет по заданному значению аргумента X значение угла в радианах в интервале от +PI/2 до -PI/2.

В приведенной ниже программе вводимое число представляется в градусах, в строке 40 оно переводится в радианы. В строке 50 значение угла в радианах используется для получения тангенса с помощью уравнения:

$$TAN(X)=\frac{SIN(X)}{COS(X)}$$

В строке 80 значение тангенса (Z) задается в качестве аргумента функции ATN для получения значения, которое печатается под заголовком ATAN(X).

Пример:

```
20 PRINT "задать угол в градусах"
30 INPUT X
40 LET Y=X*PI/180
50 LET Z=SIN(Y)/COS(Y)
60 PRINT "угол(рад)=";Y
70 PRINT "TAN(X)=";Z
80 PRINT "ATAN(Z)="; ATN(Z)
90 Goto 30

RUN
задать угол в градусах

?45
 угол(рад)=   .7853982
 TAN(X)=  1
 ATAN(Z)=   .7853982
  ---- ---
? ICy I P I
  ---- ---
Готов
```

5.3.3. Функции S I N (X), C O S (X)

в качестве аргумента задается угол,выраженный в радианах. Если угол выражен в градусах,величина аргумента ограничивается значением +-51470,нужно перевести градусы в радианы с помощью следующей формулы:

радианы = градусы .* PI/180

В примере в качестве PI используется число 3,14159265. В строке 40 приведенное значение используется для перевода в радианы.

```
 10 REM - перевести угол (X) в радианы
 11 REM - вычислить SIN и COS
 30 INPUT X
 40 LET Y=X*PI/180
 50 PRINT "радианы:";Y
 60 PRINT "SIN(Y)=";SIN(Y)
 65 PRINT "COS(Y)=";COS(Y)
 70 GOTO 30

 RUN
?10
 радианы:   .1745329
 SIN(Y)=   .1736482
 COS(Y)=   .9848078
 ---- ---
? ICy I P I
 ---- ---
Готов
```


5.3.4. Функция E X P (X)

Показательная функция возводит число "E" в степень X. EXP является функцией, обратной функции LOG, т.е.

LOG(EXP(X))=X

Пример:

```
 10 INPUT  X
 20 PRINT EXP(X)
 30 GOTO 10

 RUN
 ?4
 54.59815
 ?9.421006
 12344.99
 ?25
 .720049E 11
  ---- ---
 ? ICy I P I
  ---- ---
 Готов
```

5.3.5. Функция I N C

Вводит десятичный код клавиши, нажатой в текущий момент. Нулевое значение функции означает, что либо не нажата клавиша, либо нажата одна из клавиш:

IB/HI	IФК I ICУ I

Пример:
```
10 LETA=INC
20 IFA=0 THEN GOTO 10
30 PRINT A
40 GOTO  10
```

Таблицы кодов символов приведены в приложении 6.

5.3.6. Функция I N T (X)

Целочисленная функция вычисляет значение наибольшего целого числа, не превышающего значения аргумента.

Например:
```
PRINT INT(34.67)          PRINT INT (-5.1)
34                        -6
```

Функция может использоваться для округления чисел до ближайшего целого числа с помощью INT(X+.5)

Например:
```
PRINT INT (34.67+.5)      PRINT INT (-5+.5)
35                        -5
```

INT(X) может использоваться для округления до любого десятичного разряда с помощью следующего выражения в качестве аргумента:

(X*10^D+.5)/10^D,где D-целое число,задаваемое пользователем

Пример:

```
20 PRINT "число для округления:"
25 INPUT  A
40 PRINT "число десятичных разрядов:"
45 INPUT  D
60 LET B=INT (A*10^D+.5)/10^D
70 PRINT "A после округления =" B
80 GOTO 20
90 END

RUN
число для округления:
? 55.65842

число десятичных разрядов:
? 2

A после окру. ления = 55.66
число для округления:
? 78.375

число десятичных разрядов
?-2
A после округления =100
  ---- ---
? ICy I P I
  ---- ---
Готов
```

5.3.7. Функция L O G (X)

Функция LOG определяет натуральный логарифм заданного аргумента.
Пример:

```
10 INPUT  X
20 PRINT LOG(X)
30 GOTO 10

RUN
? 54.59815
  4
? 22026.47
  10
? 100
  4.60517
? .720049E11
  25
  ---- ---
? ICy I P I
  ---- ---
  Готов
```

Натуральные логарифмы легко перевести в логфрифмы с любым другим основачием, используя следующую формулу:

$$\log_A N = \frac{\log N}{\log A}$$

где A = нужное основание. Следующая програмна мллюстрирует перевод в десятичные логарифмы.

Пример:

```
1 REM - перевод натурадьного логарифма в десятичный
5 PRINT  "LN", "LOG"
15 INPUT X
20 PRINT LOG(X);
40 PRINT LOG(X)/LOG(10)
50 GOTO 15
60 END
Готов
RUN
LN          LOG
?4
 1.386294   .60206
?250
 5.521461   2.39794
?60
 4.094345   1.778151
```



```
  ---- ---
?ICy I P I
  ---- ---
Готов
```

5.3.8. Функция P I

Функция определяет число с точностью до седьмого знака: 3,141696.

Используется при вычислении тригонометрических функций. О применении функции PI смотрите в разделе применения тригонометрических функций.

Пример: LET A=SIN(PI)

5.3.9. Функция R N D (X)

Функция случайных чисел генерирует случайное число или совокупность случайных чисел в интервале между 0 и 1. Аргумент (X) не используется и может быть любым числом.

```
Пример:
10 PRINT "случайные числа"
30 FOR I=1 TO 5
40 PRINT RND(0)
50 NEXT I
60 END

RUN
случайные числа
 .1002502
 .9648132
 .8866272
 .6364441
 .8390198
Ост в строке 60
Готов
```

Можно генерировать случайные числа в заданном интервале. Если нужен интервал (A,B), используется выражение:

```
(B-A)*RND(0)+A
```

Следующая программа генерирует множество случайных чисел в интервале 4,6.

```
Пример:
10 REM - последовательность случайных чисел из (4,6)
20 FOR B=1 TO 5
30 LET A=(6-4)*RND(0)+4
40 PRINT A,
50 NEXT B
60 END

RUN
4.2005
5.929626
5.773254
5.272888
5.67804
Ост в строке 60
Готов
```

5.3.10. Функция S G N (X)

Функция знака SGN(X) дает значение

```
 1, если X>0
 0, если X=0
-1, если X<0
```

```
Пример:  PRINT SGN  (3.42)
   1
   PRINT SGN (-42)
   -1
   PRINT SGN (23-23)
   0
```

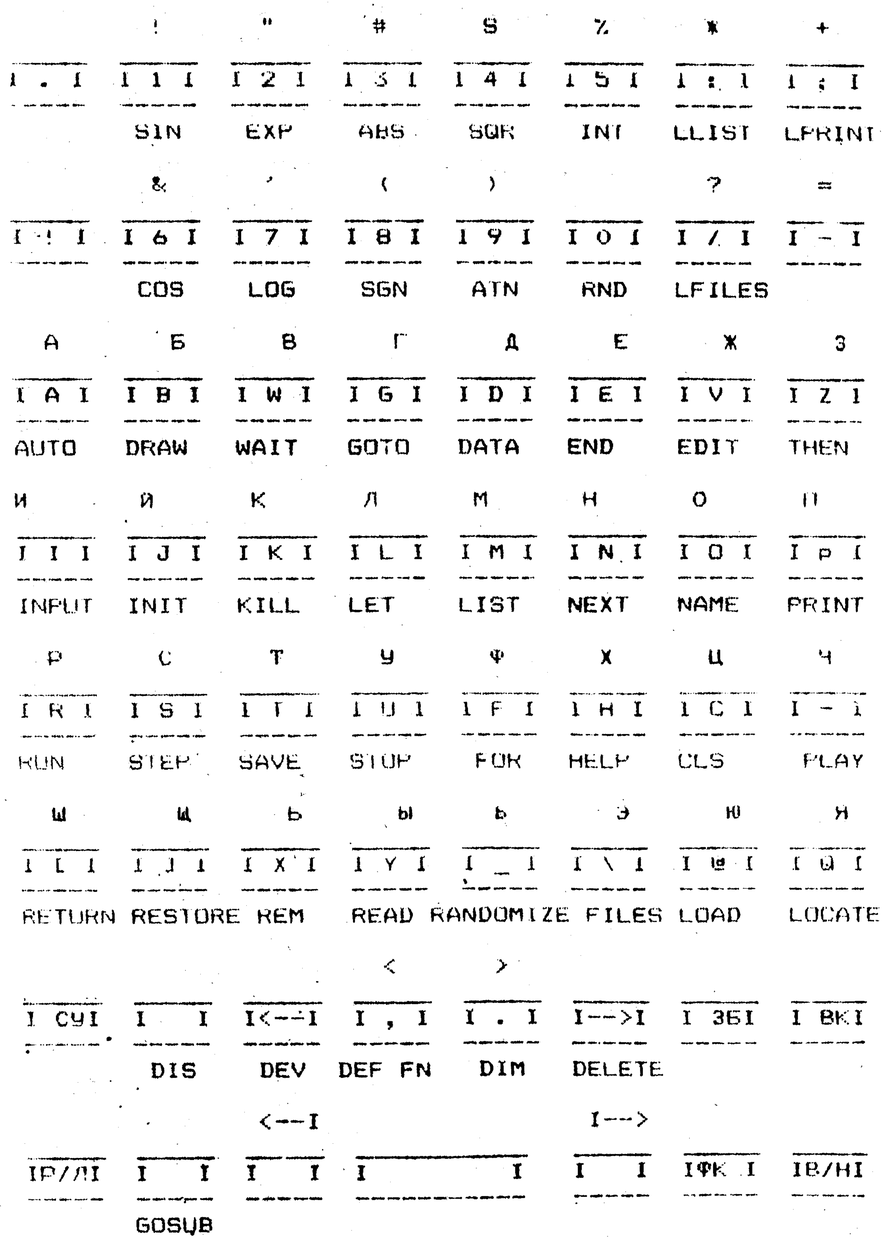

5.3.11. Функция S Q R (X)

Функция квадратного корня извлекает квадратный корень из любого положительного значения аргумента (X).

```
Пример:  10 INPUT  X
  20 LET X=SQR(X)
  30 PRINT X
  40 GOTO 10
  Готов
  RUN
  ? 16
  4
  ? 1000
  31.62278
  ? 123456789
  11111.11
  ? 25E2
  50
   _____  __
  ?I Cy I P I
   _____ ___
  Готов
```

Приложение 1

Схема клавиатуры для работы в функциональном режиме



Приложение 2

Команды микрокалькулятора

1. Команды непосредственного режима

A U T O

Функция: автоматически нумерует строки
(выход из режима по СУ/Р)
Формат : AUTO [[<начальный номер>][,<шаг>]]
Примеры: AUTO
AUTO ,20
AUTO 5,10

B L O A D

Функция: считывает в память двоичную информацию из файла, размещенного на СМП.
Формат: BLOAD "<спецификация файла>"[<стартовый адрес>] [,<длина>]

Пример: BLOAD "SM1:B.BIN"
BLOAD "SM1:B.BIN"10000,1000

B S A V E

Функция: записывает на СМП файл с двоичной информацией.
Формат: BSAVE "<спецификация файла>"<стартовый адрес>, <длина>

Пример: BSAVE "SM1:B.BIN"
BSAVE "SM1:B.BIN"10000,1000

C O N T

Функция: продолжает выполнение программы после останова.
Формат: CONT

Пример: CONT

D A T E

Функция: устанавливает или высвечивает текущую дату.
Формат: DATE [<число>-<месяц>-<год><день недели>]

Пример: DATE 1-JUN-89 MON

D E L E T E

Функция: удаляет строки из программы
Формат : DELETE [<начальный номер строки>][,<конечный номер строки>]
Примеры : DELETE 50
DELETE 70,120
DELETE 1,8191

D E V

Функция: задает имя устройства в качестве рабочего СМД
Формат : DEV ["SM<номер СМД>:"]
Примеры: DEV
DEV "SM1:"

E D I T

Функция: редактирует программу
Формат : EDIT [<номер строки>]
Пример : EDIT 150

Подфункции E D I T

^A	перемещение маркера в начало редактируемой строки
^B	перемещение маркера в начало предыдущего слова строки
^E	удаление символов строки от текущей позиции маркера до конца строки
^F	перемещение маркера в начало следующего слова
^X	перемещение маркера в конец строки
^H	вызов подсказки
I-->	режим вставки/замены
<--I	смыкание строки влево до позиции маркера
I -->I	перемещение маркера на одну позицию вправо
I<-- I	перемещение маркера на одну позицию влево
I ! I	переход к предыдущему оператору

I ! I	переход к следующему оператору
I ! I	переход к предыдущему оператору в буфере команд
I ! I	переход к следующему оператору в буфере команд
I BK I	выход из режима редактирования

F I L E S

Функция: распечатывает справочник СМД
Формат : FILES ["SM<номер СМД>:"]
Примеры: FILES
FILES "SM1:"

I N I T

Функция: инициализирует СМД
Формат : INIT ["SM<номер СМД>:"]
Примеры: INIT "SM1:"
INIT

K I L L

Функция: удаляет из СМД указанный файл
Формат : KILL "[SM<номер СМД>:]<имя файла>.<тип>"
Пример : KILL "SM0:TEXT.BAS"

L I S T

Функция: печатает текст программы
Формат : LIST [<начальный номер строки>][,<конечный номер строки>]
LIST "< спецификация файла >"[<начальный номер строки>]
Примеры : LIST
LIST 20
LIST 30,50
LIST 20,
LIST 50,
Функция: печатает текст программы из файла, имеющего ASCII формат и размещенного на СМД 039 или СМД 039.

L O A D

Функция: Загружает в память программу из СМД, хранящуюся во внутреннем или ASCII формате.
Формат : LOAD "[SM<номер СМД>:]<имя вводимого файла>

[.<тип>]"[,A][,R]

Примеры: LOAD "SM0:TEXT.BAS"
LOAD "SM1:BASIC"
LOAD "SM1:PROG1.BAS",A

M E M

Функция: выдает распределение памяти
Формат: MEM
Пример: MEM

N A M E

Функция: переименовывает файл на СМЛ
Формат : NAME "<старое имя>"AS"<новое имя>"
Пример : NAME "SM1:OLD.BAS"AS"NEW.BAS"

N E W

Функция: удаляет из памяти все программные строки, переменные и массивы.
Формат: NEW

Пример: NEW

P A T C H

Функция: позволяет проверять и изменять содержимое любой ячейки памяти программы в машинных кодах.
Формат: PATCH
на экране печатается "*".

Операторы и команды PATCH :

xxxxxx - адрес ячейки (6-значное восьмеричное число).
/или A / - оператор вывода восьмеричных слов, A - адрес открываемого слова.
\или A \ - оператор вывода восьмеричных байтов, A - адрес открываемого байта.
<ВК> или
M <ВК> - закрывает текущую открытую ячейку. Значение M заменяет содержимое ячейки.
или M - закрывает текущую открытую ячейку и открывает следующую. Значение M заменяет содержимое ячейки.
или M - закрывает текущую открытую ячейку и открывает предыдущую. Значение M заменяет содержимое ячейки.
@ или M @ - закрывает текущую открытую ячейку и открывает ячейку по абсолютному адресу, которым является содержимое текущей ячейки. Значение M заменяет содержимое текущей открытой ячейки перед тем, как она будет закрыта (И использоваться как абсолютный адрес).

A G - запуск программы пользователя с абсолютного адреса A.
E - окончание работы с PATCH.

R E N U M

Функция: перенумерация программных строк.
Формат: RENUM <номер X1>,<номер X2>,приращение
X1 - начальный новый номер первой строки;
X2 - старый номер, с которого начинать перенумерацию строк;
приращение -разность между двумя соседними новыми номерами.

Пример:
```
10 CLS
20 PRINT "****": GOTO 30
30 END

RENUM 15,20,100

10 CLS
15 PRINT "****": GOTO 115
115 END
```

R U N

Функция: выполняет программу
Формат : RUN
Пример : RUN

S A V E

Функция: сохраняет программу на СМЛ во внутреннем или ASCII формате
Формат : SAVE "[SM<номер СМЛ>:]<имя файла>[.<тип>]"[,A]
Примеры: SAVE "SM0:TEXT.BAS"
SAVE "SM1:BASIC"
SAV "SM1:PROG1.BAS"

S Y S T E M

Функция: передает управление системному загрузчику.
Формат: SYSTEM

Пример: SYSTEM

T I M E

Функция: устанавливает или высвечивает текущее время.
Формат: TIME [<чч>:<мм>:<сс>]
<чч> - часы,

<mm> - минуты,
<cc> - секунды.

Пример: TIME 21:00:15

U S R

Функция: резервирует сегмент памяти для пользователя.
Формат: USR [<длина сегмента>]

Пример: USR 100

2. Команды программного режима

C A L L

Функция: передает управление програмне в машинных кодах, находящейся в памяти по абсолютному адресу.
Формат: CALL <абсолютный адрес>[<имя>]
<имя> - имя переменной или массива.
Пример: CALL 10000,A

C L O S E

Функция: закрывает файл на внешнем устройстве
Формат: CLOSE # <номер файла>
Пример: CLOSE #1

C L S

Функция : очищает экран ЖКИ и переводит курсор в крайнюю левую позицию.
Формат : CLS
Пример : CLS

D A T A

Функция: вводит в программу числовые константы, с которыми оператор READ последовательно связывает имена переменных
Формат : DATA <число> [,<число>]...
Пример : DATA 5,6,7,8

D E F

Функция: определяет функции и выражения как отдельную функцию
Формат : DEF FN[<буква>][(<параметр>)]=<выражение>
Примеры: DEF FNX(S)=S^2+4
DEF FNZ(X)=X^2
DEF FNA(X)=4+2

D I M

Функция: резервирует память для формирования массива
Формат : DIM <имя масоива>(<числовое значение>
[,<числовое значение>])
Примеры: DIM A(10)
DIM B(5,2)

D I S

Функция: восстанавливает служебную строку
Формат : DIS
Пример : DIS

D R A W / A

Функция: высвечивает прямоугольник, заданный координатами его диагонали.
Формат : DRAW A <начальная координата X>,
<начальная координата Y>,
<диагональная координата X>,
<диагональная координата Y>
Примеры : DRAW A0,0,30,60

D R A W / C

Функция: высвечивает окружность по заданным координатам центра и радиусу.
Формат : DRAW C <координата X центра>, <координата Y центра>, <радиус>

Пример : DRAW C60,32,10

D R A W / D

Функция: высвечивает отрезки прямых между последовательностью заданных точек.
Формат : DRAW D <начальная координата X>, <начальная координата Y>, <координата X>, <координата Y>
Пример : DRAW D34,34,67,50

D R A W / E

Функция: гасит точку (линию) в заданном положении.
Формат : DRAW E <начальная координата X>, <начальная координата Y>, <координата X>, <координата Y>
Пример : DRAW E34,34,67,50

D R A W / G

Функция: вычерчивание вертикальных или горизонтальных полос заданной ширины.
Формат : DRAW G <направление штриховки>,
<протяженность по оси X>,
<протяженность по оси Y>,
<расстояние между линиями>
Пример : DRAW G1,20,20,2

D R A W / H

Функция: высвечивает точку в заданной позиции
Формат : DRAW H <координата X>, <координата Y>,
<переменная>
Пример : DRAW HX,Y

D R A W / I

Функция: вычерчивает прямую линию, координаты которой заданы относительно.
Формат : DRAW I <приращение по X>,<приращение по Y>
Примеры: DRAW I20,10,-0,-30
DRAW I-10,0

D R A W / O

Функция: определяет текущую точку экрана.
Формат : DRAW O <координата X>, <координата Y>
Примеры: DRAW OX,Y
DRAW O60,32

D R A W / X

Функция: высвечивает оси координат.
Формат : DRAW X <направление оси координат>, <размер деления оси>, <количество делений>
Пример : DRAW X1,5,5

E N D

Функция: последний оператор программы
Формат : END
Пример : END

F O R - T O - S T E P / N E X T

Функция: операторы FOR и NEXT используются для обозначения начальной и конечной точек цикла программы
все операторы,стоящие между оператором FOR и соответствующим ему оператором NEXT, будут циклически выполнятся в соответствии с условиями, заданными в операторе FOR
Формат : FOR <имя переменной>=<числовое выражение>
TO <числовое выражение>
[STEP <числовое выражение>] NEXT <имя переменной>
Примеры : FOR I=0 TO 9

```
NEXT I
FOR  A=1 TO  9 STEP 2
PRINT A
NEXT A
```

G E T

Функция: читает значение переменной в ASCII-формате с СМП
Формат: GET # <номер файла>, <имя переменной>
Пример: OPEN #1,1," B.BAS "
GET #1,G1

G O S U B / R E T U R N

Функция: осуществляет вход в подпрограмму и возвращение из подпрограммы
Формат : GOSUB < номер строки >
RETURN
Пример : GOSUB 200
RETURN

G O T O

Функция: осуществляет безусловный переход к строке с указанным номером
Формат : GOTO <номер строки>
Пример : GOTO 210

I F

Функция: обеспечивает условный переход в программе
Формат: IF <условное выражение> [THEN <оператор>]
[THEN <номер строки>]
[GOTO <номер строки>]
Примеры: IF A>B THEN 10 '
IF A>4 GOTO 50
IF X>=70 THEN GOSUB 100

I N P U T

Функция: вводит данные с клавиатуры во время выполнения программы
Формат : INPUT < имя переменной > [,<имя переменной>]
Примеры: INPUT X
INPUT A,B,C

L E T

Функция: присваивает значение переменной

Формат : LET <имя переменной>=<числовое выражение>
Примеры: LET A=2
LET X=A+B
LET X=Y*2

L O C A T E

Функция: определяет позицию курсора
Формат : LOCATE <координата X>,<координата Y>
Пример : LOCATE 60,32

O P E N

Функция: присваивает номер поименованному файлу и разрешает доступ к нему
Формат: OPEN # <номер файла>, I/O, "<спецификация файла>"
Пример: OPEN #1,O," SM1:A"

P A I N T

Функция: заполняет заданную область экрана указанной маской.
Формат: PAINT<номер столбца>,<номер строки>,<номер маски>
<номер столбца>,<номер строки>- координаты внутренней точки заполняемой области.
Соответствие маски и рисунка приведено в Приложении 5.

Пример: PAINT 60,30,48

P L A Y

Функция : подает звуковые сигналы
Формат : PLAY <тон>,<длительность>
Пример : FOR I=1 TO 40
PLAY 1,20
NEXT I

P R I N T

Функция: выводит (печатает) данные на экран
Формат : PRINT
PRINT <выражение> [,<выражение>]...
PRINT "<строка символов>"[,"<строка символов>"]...
Примеры: PRINT A,B
PRINT A+B
PRINT "результат"

P U T

Функция: записывает значение переменной в ASCII-формате на СМП

Формат: PUT # <номер файла>, <имя переменной>
Пример: OPEN #1,O," SM1:A"
PUT #1, SIN (I)

R A N D O M I Z E

Функция: позволяет получить ряд случайных чисел в программе, используя функцию RND
Формат : RANDOMIZE
Пример : RANDOMIZE

R E A D

Функция: читает данные, которые были определены в DATA и последовательно связывает их со своими переменными
Формат : READ <имя переменной> [,<имя переменной>]...
Пример : READ S,T

R E M A R K

Функция: определяет комментарии в программе пользователя
Формат : REMARK - <комментарий>
REM - <комментарий>
Пример : REM - вычисление факториала X

R E S T O R E

Функция: осуществляет повторное использование констант оператора DATA, который имеет самый низкий номер строки в программе
Формат : RESTORE
Пример : RESTORE

S T O P

Функция: останавливает выполнение программы
Формат : STOP
Пример : STOP

T R O F F

Функция: отменяет режим трассировки программы.
Формат: TROFF

Пример: TROFF

T R O N

Функция: устанавливает режим трассировки программы.
Формат: TRON

Пример: TRON

WAIT

Функция: приостанавливает выполнение программы
Формат : WAIT <переменная>
Аргумент <переменная> определяет величину задержки
Пример : WAIT 3530

3. Встроенные функции

A B S

Функция: абсолютное значение
Формат : ABS(<числовое выражение>)
Пример : ABS(-10.5)

A T N

Функция: обратная тригонометрическая функция от TAN(X)
Формат : ATN(<числовое выражение>)
Пример : ATN(A/100)

C O S

Функция: тригонометрическая функция косинуса COS(X).
Формат : COS(<числовое выражение>)
Пример : COS(A*10)

E X P

Функция: показательная функция
Формат : EXP(<числовое выражение>)
Пример : EXP(1)

I N C

Функция: выдача кода нажатой клавиши
Формат: INC
Пример: 10 LET A=INC: IF A=0 GOTO 10
20 PRINT A

I N T

Функция: целочисленная функция. вычисляет значение наибольшего числа, не превышающее значения аргумента
Формат : INT(<числовое выражение>)
Пример : INT(3.14)

L O G

Функция: натуральный логарифм
Формат : LOG(<числовое выражение>)
Пример : LOG(2.71828)

R N D

Функция: генерирует случайное число или совокупность

случайных чисел в интервале [0,1]
Формат : RND(<любое число>)
Пример : RND(0)

S G N

Функция: знак числового выражения
Формат : SGN(<числовое выражение>)
Пример : SGN(-1)

S I N

Функция: тригонометрическая функция синуса SIN(X)
Формат : SIN(<числовое выражение>)
Пример : SIN(A/B)

S Q R

Функция: квадратный корень
Формат : SQR(<числовое выражение>)
Пример : SQR(30)

Приложение 3.

Сообщения об ошибках

1. Формат сообщений

Бейсик осуществляет проверку операторов программы и вводимых данных и для каждой обнаруженной ошибки печатает соответствующее сообщение об ошибке
Сообщения об ошибках имеют следующий формат:

ош XXX стр YYY

где XXX - код ошибки,
YYY - номер строки, в которой произошла ошибка
Если YYY=0, это означает, что ошибка произошла в режиме команд, т.е. в последней вводимой строке.

2. Ошибки

Коды ошибок с 0 по 64 указывают на неустранимые ошибки, выполнение программы прекращается после печати сообщения об ошибке.
Неустранимые ошибки приведены в табл. 1.

Коды ошибок с 89 по 127 указывают на устранимые ошибки; после печати сообщения об ошибке выполнение программы продолжаетоя.
Устранимые ошибки приведены в табл. 2.

таблица 1

Неустранимые ошибки

код ошибки	содержание ошибки
0	переполнение памяти, отведенной пользователю
1	нераспознаваемый оператор
2	недопустимый оператор GOTO или GOSUB
3	недопустимый знак, ограничивающий оператор (обычно в случае неправильно сформированного оператора, который вызывает преждевременное окончание действия оператора)
4	оператор RETURN без соответствующего оператора GOSUB
5	неправильно сформирован индекс
6	индекс не попадает в интервал от 0 до 255 или превышает максимум, установленный программой
7	несоответствие скобок в операторе
8	недопустимый оператор LET
9	недопустимый знак операции отношения в операторе IF
10	недопустимый оператор IF
11	недопустимый оператор PRINT
12	слишком длинная вводимая строка (превышает 80 знаков)
13	неправильная размерность в операторе DIM
14	в памяти недостаточно места для массива
15	неправильно сформирован оператор DEF
16	недопустимый номер строки или значение размерности
17	оператор DIM для ранее описанного или использованного элемента

Продолжение табл. 1

код	содержание ошибки
18	неправильная переменная в списке оператора INPUT
19	неправильная переменная в списке оператора READ
20	данные в списке оператора READ исчерпаны
21	неправильный формат оператора DATA
22	недопустимый оператор FOR
23	FOR не сопровождается соответствующим оператором NEXT
24	оператор NEXT без оператора FOR
25	несоответствие кавычек в операторе
27	неправильно сформировано выражение (опущен порядок числа в формате E)
28	неверный синтаксис оператора PRINT
29	неверная функция оператора DRAW
30	ошибка в задании параметров
31	неверная нота в операторе PLAY
32	неверная функция оператора PLOT (PLOTR)
33	недопустимый вызов функции RENUM
34	недопустимая ссылка
50	отсутствует ПЗУ в блоке расширения
51	блок расширения не готов
53	СМП не готов
55	ошибка ввода-вывода
56	защита записи
57	переполнение файла
58	несовпадение контрольных сумм

Продолжение табл. 1

код ошибки	содержание ошибки
59	нет места в СМП
60	не найден переименовываемый файл
61	неверный формат оператора NAME
62	файл не найден
63	синтаксическая ошибка в имени файла

Таблица 2

Устранимые ошибки

код ошибки	содержание ошибки
120	недопустимые знаки при вводе
121	введено недостаточно данных по оператору INPUT
122	введено олишком много данных по оператору INPUT
123	несуществующая переменная
124	число слишком велико для фиксации (вероятно, комбинация индексов превышает диапазон)
125	переполнение или заем при делении/умножении
126	квадратный корень из отрицательного числа
127	логарифм отрицательного числа или нуля; переполнение при вычислении exp

Приложение 4

Примеры программ

1. Построение кривых

1.1. Построение окружности

При использовании оператора DRAW/C может быть вычерчена окружность. При этом достаточно определить координаты ее центра и радиус.

Программа, с помощью которой производится вычерчивание окружности радиусом 25 и с центром в координатах (50,32):

```
10 DRAW C50,32,25
```

Теперь вычертим концентрические окружности радиусами 10, 20 и 30 с центром в координатах (50,32).

```
 5 CLS
10 FOR R=1 TO 3
20 DRAW C50,32,R*10
30 NEXT R
```

1.2. Построение синусоиды

В программе построения синусоиды используется построение осей координат, нанесение делений шкалы и вычерчивание синусоиды с помощью оператора DRAW/X.

Значения шкал могут быть расположены в вертикальном направлении с помощью оператора PRINT.

```
10 CLS
20 FOR I=0 TO 2 STEP 2
30 DRAW O20,32
40 DRAW XI,5,4 :NEXT I
50 DRAW O20,32
60 DRAW X1,10,8
70 GOSUB 200
80 GOSUB 400
90 GOSUB 600
100 END
200 LET A=28
205 FOR I=90    TO 360  STEP 90
```

```
210 DRAW 0A,25
220 PRINT <U3; Y3> 1;
225 LET A=A+25
226 IF  I=90 THEN LET A=A-6
230 NEXT I
240 RETURN
400 FOR I=-1 TO 1 STEP 1
410 DRAW 02,30-I*20
420 PRINT I
430 NEXT I
440 RETURN
600 FOR I=0 TO 360 STEP 4.5
610 LET X=20+I*(2/9)
620 LET Y=32+20*SIN(PI/180*I)
630 DRAW HX,Y
640 NEXT I
650 RETURN
```

2. Построение линейных графиков

Поскольку для построения графика требуется задание координат, то их нужно определить прежде всего.

Начало координат может быть помещено в положение (X,Y) с помощью оператора:

```
DRAW    0X,Y
```

Вычерчивание осей координат производится с помощью операторов:

```
DRAW    X0,5,10
DRAW    X1,5,10
DRAW    X2,5,10
DRAW    X3,5,10
```

числа, следующие после X, указывают на направление осей.

0: вычерчивается ось Y из начала координат вверх
1: вычерчивается ось X из начала координат вправо
2: вычерчивается ось Y из начала координат вниз
3: вычерчивается ось X из начела координат влево

Следующим числовым значением (5) определяется расстояние между метками шкалы (в точках), число 10 определяет количество меток шкалы.

Эти операторы используются в качестве примера в следующей программе:

```
 5    CLS
10    FOR  I=0   TO 3
20    DRAW 060,32
30    DRAW X1,5,6
40    NEXT   I
```


При выполнении этой программы вычерчиваются оси координат с началом в точке (60,32), показанные на рис.1.

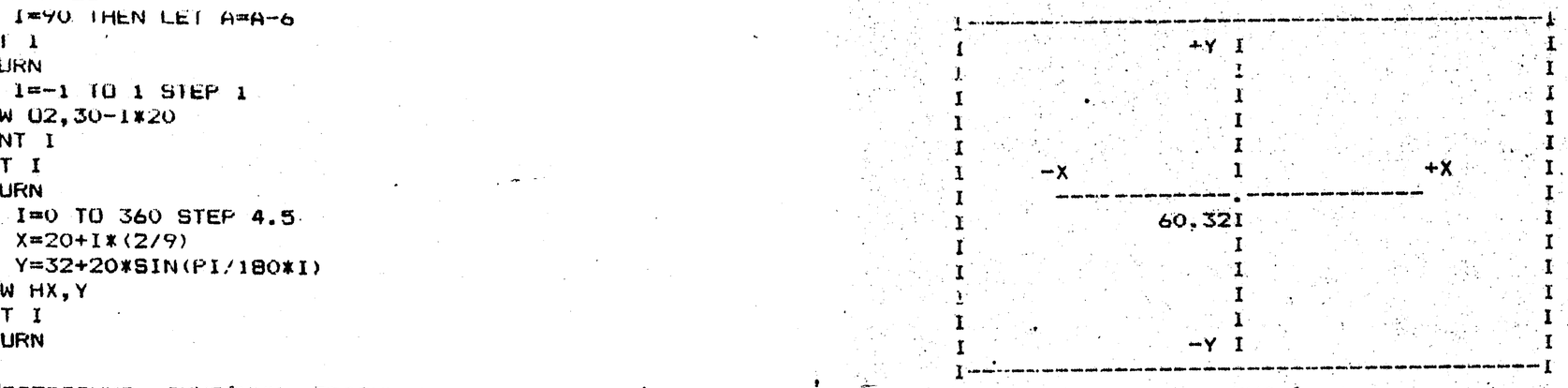


рис.1.

Линейные графики часто используются при исследовании процессов, протекающих во времени. Рассмотрим программу, с помощью которой строится линейный график средней температуры за месяц.

Предположим, что средняя температура за месяц в определенном городе изменялась в соответствии с табл.1.
Эти данные вводятся в программу с помощью оператора DATA. Затем, с помощью программы производится построение графика средней температуры за месяц. Считывание данных производится с помощью оператора READ.

таблица 1.

месяц	температура	месяц	температура	месяц	температура
январь	11,5	май	19,8	сентябрь	22,3
февраль	9,8	июнь	23,4	октябрь	18,5
март	13,7	июль	26,6	ноябрь	15,9
апрель	18,3	август	28,2	декабрь	14,7

```
10   CLS
20   DRAW 030,50
30   DRAW X0,10,4
35   DRAW 030,50
40   DRAW X1,5,12
50   FOR  I=1 TO 3
60   LOCATE 15,1*10
70   PRINT  40-I*10
80   NEXT   I
90   FOR    I=1 TO 12
100  LET X=30+(I-1)*5
110  READ  A
120  LET Y=INT(50-A)
130  IF I=1 THEN 160
140  DRAW DP,0,X,Y
150  NEXT I
160  LET P=X: LET Q=Y
170  DATA  11.5,9.8,13.7,18.3,19.8,23.4
175  DATA  26.6,28.2,22.3,18.5,15.9,14.7
180  END
```

3. Построение гистограмм

3.1. Построение гистограмм с использованием символов

Принцип построения гистограммы заключается в том, что на экране дисплея высвечивается рисунок в виде столбиков, длина которых пропорциональна значению N. Поскольку на экране дисплея вмещается только 20 цифр, то для того,чтобы высвечивать линейные величины, превышающие число 20, необходимо использовать масштабирование.

Если вы выполните следующую программу и присвоите N значения от 1 до 20, то на экране высветится линия длиной в N символов. Этот столбик будет непрерывным.

```
10   CLS
20   INPUT N
25   LOCATE 0,32
30   FOR I=1    TO N
40   PRINT "_";
50   NEXT   I
60   END
```

3.2. Построение гистограмм с помощью графиков

Приведенную выше программу можно переделать таким обра-

зом, чтобы вместо символов использовались точки. Следующая программа является таким примером .

С помощью этой программы по горизонтали может быть высвечено 120 точек, при этом каждая точка по длине в шесть раз короче символа.

```
10   CLS
20   INPUT N
30   FOR I=0 TO N-1
40   DRAW HI,10
50   NEXT  I
60   END
```

Получаемые с помощью этой простой программы гистограммы имеют вид прямых линий. Можно написать программу в которой вместо горизонтальных линий будут высвечиваться столбики.

```
10   CLS
20   INPUT N
30   DRAW A0,32,N-1,26
40   END
```

Получаемые с помощью этой простой программы столбики светятся монотонно. Можно написать программу,в которой столбик будет выглядеть в виде определенного узора. Такие узоры можно реализовать путем внесения изменений и добавлений в выше приведенную программу.

1) Горизонтальная штриховка столбика:

```
35   DRAW  00,26
36   DRAW  G1,N-1,6,2
```

2) Вертикальная штриховка столбика:

```
35   DRAW  00,26
36   DRAW  G2,N-1,6,2
```

3.3. Построение круговых диаграмм

Для построения окружности используется команда DRAW C. Однако, для построения круговой диаграммы необходимо провести линии, определяющие сектора, соответствующие определенным величинам.

Ниже в качестве примера показана программа,которая позволяет разделить окружность на 5 частей.

```
10   CLS
20   DRAW C50,32,30
30   LET S=0: LET T=0
40   FOR I=1     TO 5: READ A: LET T=T+A:NEXT I
50   RESTORE: LET A=0
60   FOR I=1     TO 5: LET A1=A
70   READ A: LET S=S+A
80   LET X=50+INT(30*COS(2*PI*S/T))
90   LET Y=32-INT(30*SIN(2*PI*S/T))
100  DRAW D50,32,X,Y
110  LET A2=(A1+A)/2 :  GOSUB 200
115  NEXT I
120  END
130  DATA 100,200,300,400,500
200  LET A3=COS(2*PI*S/(T+A2))
205  LET X=84
206  IF A3<0 THEN LET X=5
210  LET  Y=25-INT(22*SIN(2*PI*S/(T+A2)))
220  DRAW UX,Y
230  PRINT  I*100
240  RETURN
```

3.4 Два примера построения гистограмм

Гистограммы часто используются для наглядного представления соотношения некоторых данных.

Однако необходимо, чтобы наряду с сопоставительной наглядностью столбики гистограммы правильно отражали количественно отображаемую величину. Поэтому здесь важно правильно выбрать масштаб, чтобы гистограмма хорошо уложилась на экране.

Существует много путей построения графиков в зависимости от их применения. В основном, используются такие гистограммы,в которых одним столбиком выражается количественное значение одной величины (например, когда иллюстрируется количество различных изделий). Бывают гистограммы, в которых одним столбиком иллюстрируется как общее количество, так и какие-нибудь детали (например,соотношение общего количества и конкретного изделия).

Напишем программы, с помощью которых с использованием следующего примера производится построение таких гистограмм двух типов.

Пример данных: количество автомобилей двух типов а и в, изготавливаемых на одном производстве,приведено в табл.2.

Таблица 2.

	1980	1981	1982
автомобили а	48200	57200	67200
автомобили в	39200	31100	27500

Программа построения гистограммы, в которой одним столбиком отображается количественное значение только одной величины.

Столбик гистограммы, отображающий производство автомобилей типа а - светлый, столбик, отображающий производство автомобилей типа в - темный.

```
10   CLS
20   FOR I=0 TO 2
30   LOCATE 12,10+I*16
40   PRINT 80+I
50   FOR J=1 TO 2
60   READ A
70   LET Y=1*16+(J-1)*8+10
80   DRAW A 30,Y+4,30+INT(A/1000),Y
90   IF J=1 THEN GOTO 120
100  DRAW D30,Y
110  DRAW G1,INT(A/1000),4
120  NEXT J:     NEXT I
130  DATA 48200,39200,57200,31100,67200,27500
140  END
```

Программа построения гистограммы, в которой одним столбиком одновременно отображается общее количество и детали.

```
10   CLS
20   FOR I=0 TO 2
30   LOCATE 12,10+I*16
40   PRINT 80+I
50   READ  A,B
60   LET Y=I*16+10
70   DRAW A30,Y+8,30+INT(A/1000),Y
75   DRAW 030,Y
80   DRAW A30,Y+8,30+INT(B/1000),Y
85   DRAW 030,Y
90   DRAW G1,INT(B/1000),8
100  NEXT I
110  DATA  48200,39200,57200,31100,67200,27500
120  END
```

4. Изображение движущихся объектов

4.1. Создание движущихся рисунков

Если вы выполните следующую программу, метка '*' будет передвигаться слева направо и справа налево.

```
10   CLS
20   FOR I=0 TO 14 ---
                     1
30   LOCATE I*6,8    1   движение слева направо
                     1
40   PRINT <S2,2>" *"1
                     1
50   NEXT  I ---------
```

```
60   FOR I=14   TO 0 STEP-1 ---
                                I
70   LOCATE I*6,8               I
                                I
80   PRINT <S2,2>"*      "     I  движение справа
                                I        налево
90   NEXT  I                    I
                                I
100  GOTO  20      ------------
```

Как показано в программе, при изменении управляющей переменной I в строке 20 от 0 до 17,с помощью оператора строки 30 положение курсора изменяется от (0,8) до (114,8).Метка '*' высвечивается на экране путем вычерчивания " *", в то время,как метка '*', высветившаяся перед этим, стирается пробелом.

При повторении этой процедуры создается впечатление, что она перемещается слева направо. Операторы, расположенные в строках 60-90, аналогичным образом перемещают метку справа налево.

Вертикальное положение теоретически выполняется аналогичным образом. Однако здесь не может происходить одновременное высвечивание и стирание,и программа приобретает следующий вид:

```
10   CLS

20   FOR  I=0   TO 7      ------------
                                      I
30   LOCATE 12,I*8: PRINT        "*" I  движение
                                      I  вниз
40   IF  I=0 THEN 60                  I
                                      I
50   LOCATE 12,(I-1)*8: PRINT" "      I
                                      I
60   NEXT  I                ----------
70   FOR  I=7 TO 0 STEP -1  ---------
                                      I
80   LOCATE 12,I*8: PRINT "*";        I
                                      I
90   IF  I=7  THEN  110               I  движение
                                      I     снизу
100  LOCATE 12,(I+1)*8: PRINT " "; I  I     вверх
                                      I
110  NEXT  I                ---------
120  GOTO  20
```

4.2. Изменение скорости движения

Предыдущая программа обеспечивает довольно быстрое перемещение метки "*". Скорость ее движения управляется оператором FOR/NEXT.

Выполним программу,полученную путем добавления к предыдущей программе следующей строки:

```
45  FOR  J=1    TO 50:  NEXT J
```

При использовании этого оператора скорость перемещения слева направо становится несколько меньше. Скорость увеличивается путем уменьшения конечного значения оператора FOR/NEXT, а уменьшается путем увеличения этого значения.

Управление скоростью производится путем добавления этого оператора в нужную часть программы.

4.3. Перемещение точки по прямой

Для перемещения точки по прямой необходимо выполнить следующую программу:

```
10   CLS          ---------
                           I
20   FOR I=0 TO 119        I
                           I
30   DRAW HI,32            I    движение слева
                           I    направо.
40   NEXT I       ---------
70   FOR  I=119 TO 0    STEP-1 -----
                                   I
90   IF  I=119  THEN 110           I       движение
                                   I       справа
100  DRAW E1+1,32,1+1,32           I       налево
                                   I
110  NEXT  I                ------------

130  CLS          ---------
                           I
140  FOR I=0   TO 63       I  движение сверху
                           I       вниз
150  DRAW H60,I            I
                           I
160  NEXT I       ---------
190  FOR I=63 TO 0 STEP -1 -----
                                   I
210  IF I=63 THEN 230              I    движение
                                   I     снизу
220  DRAW E60,I+1,60,I+1           I     вверх
                                   I
230  NEXT I                 ------------
240  GOTO 20
```

4.4. Перемещение точки по кривой траектории

Следующая программа обеспечивает повторное перемещение точки по траектории, как показано в примере выполнения.

```
10   CLS
20   LET N=1    : LET P=0
30   FOR I=0 TO 90      STEP 3
40   LET X=P+N: LET Y=30-25*ABS(COS(PI/180*I))
50   IF N=1 THEN 70
60   DRAW BP,U,P,U: GOTO 80
70   DRAW BX,Y
80   LET P=X:  LET U=Y
90   NEXT I
100  LET P=P+N:LET N=-N
110  GOTO 10
```

5. Пример использования оператора CALL для считывания показаний таймера.

Определить свободную область памяти

```
 USR 200
 37276
```

Ввести программу считывания показаний таймера в машинных кодах, используя команду PATCH.

Программа на ассемблере		в машинных кодах	
	MOV R0,-(R6)	010046	
	MOV R1,-(R6)	010146	
	MOV R2,-(R6)	010246	
	MOV R3,-(R6)	010346	
	MOV R4,-(R6)	010446	
	MOV R5,-(R6)	010546	
	MOV #165010,R5	012705	165010
	MOV #3,R2	012702	000003
BEG:	MOV (R5),R1	011501	
	BIC #1777400,R1	042701	177400
	ASR R1	006201	
	MOV R3,R0	010300	
	MOV R2,-(R6)	010246	
	MOV R3,-(R6)	010346	
	TRAP 36	104436	
	MOV (R6)+,R3	012603	
	MOV (R6)+,R2	012602	
	CMP #1,R2	022702	000001
	BEQ EN	001404	
	CMP -(R5),-(R5)	024545	
	ADD #6,R3	062703	000006
	SOB R2,BEG	077221	
EN:	MOV (R6)+,R5	012605	
	MOV (R6)+,R4	012604	
	MOV (R6)+,R3	012603	
	MOV (R6)+,R2	012602	
	MOV (R6)+,R1	012601	

	MOV (R6)+,R0	012600	
	RTS PC	000207	

Затем выполнить следующую программу на Бейсике:

```
10 DIM A(3)
20 CALL #37276,A(1)
30 FOR J=0 TO 2
40 PRINT A(J)
50 NEXT J
60 END
```

где A(0) - часы,
A(1) - минуты,
A(2) - секунды,
число 36500 - адрес программы в машинных кодах.

Приложение 5

Таблица масок для оператора Р А Й Н

0	1	2	3	4
11111111	00000000	01110111	10000000	01110111
11111111	00000000	11111111	00000000	11011101
11111111	00000000	11011101	00010001	00000000
11111111	00000000	11111111	00000000	11111111
11111111	00000000	01110111	10000000	01110111
11111111	00000000	11011101	00000000	11011101
11111111	00000000	11111111	00000000	11111111

Приложение 6

Таблицы кодов символов

Таблица 1

Специальные символы

восьмер. код символа	десятич. код символа	символ
0	0	
1	1	
2	2	
3	3	
4	4	
5	5	
6	6	
7	7	
10	8	
11	9	
12	10	пс
13	11	
14	12	
15	13	вк
16	14	рус
17	15	лат
20	16	
21	17	
22	18	
23	19	
24	20	

Продолжение табл. 1

восьмер. код символа	десятич. код символа	символ
25	21	
26	22	
27	23	
30	24	
31	25	
32	26	
33	27	
34	28	
35	29	
36	30	
37	31	
40	32	пробел
41	33	!
42	34	"
43	35	#
44	36	$
45	37	%
46	38	&
47	39	'
50	40	(
51	41	)
52	42	*
53	43	+
54	44	,
55	45	-
56	46	.
57	47	/
60	48	0
61	49	1
62	50	2
63	51	3
64	52	4
65	53	5
66	54	6
67	55	7
70	56	8
71	57	9
72	58	:
73	59	;
74	60	<
75	61	=
76	62	>
77	63	?

Таблица 2

Символы латинокого регистра

Восьмеричный код символа	Десятичный код символа	символ	Восьмеричный код символа	Десятичный код символа	символ
100	64	@	140	96	`
101	65	A	141	97	a
102	66	B	142	98	b
103	67	C	143	99	c
104	68	D	144	100	d
105	69	E	145	101	e
106	70	F	146	102	f
107	71	G	147	103	g
110	72	H	150	104	h
111	73	I	151	105	i
112	74	J	152	106	j
113	75	K	153	107	k
114	76	L	154	108	l
115	77	M	155	109	m
116	78	N	156	110	n
117	79	O	157	111	o
120	80	P	160	112	p
121	81	Q	161	113	q
122	82	R	162	114	r

Продолжение табл. 2

Восьмеричный код символа	Десятичный код символа	символ	Восьмеричный код символа	Десятичный код символа	символ
123	83	S	163	115	s
124	84	T	164	116	t
125	85	U	165	117	u
126	86	V	166	118	v
127	87	W	167	119	w
130	88	X	170	120	x
131	89	Y	171	121	y
132	90	Z	172	122	z
133	91	[	173	123	{
134	92	\	174	124	|
135	93	]	175	125	}
136	94	^	176	126	~
137	95	_	177	127	36

Таблица 3

Символы русского языка

Восьмеричный код символа	Десятичный код символа	символ	Восьмеричный код символа	Десятичный код символа	символ
100	64	ю	140	96	Ю
101	65	а	141	97	А
102	66	б	142	98	Б
103	67	ц	143	99	Ц
104	68	д	144	100	Д
105	69	е	145	101	Е
106	70	ф	146	102	Ф
107	71	г	147	103	Г
110	72	х	150	104	Х
111	73	и	151	105	И
112	74	й	152	106	Й
113	75	к	153	107	К
114	76	л	154	108	Л
115	77	м	155	109	М
116	78	н	156	110	Н
117	79	о	157	111	О
120	80	п	160	112	П
121	81	я	161	113	Я
122	82	р	162	114	Р
123	83	с	163	115	С

Продолжение табл. 3

Восьмеричный код символа	Десятичный код символа	символ	Восьмеричный код символа	Десятичный код символа	символ
124	84	т	164	116	Т
125	85	у	165	117	У
126	86	ж	166	118	Ж
127	87	в	167	119	В
130	88	ь	170	120	Ь
131	89	ы	171	121	Ы
132	90	з	172	122	З
133	91	ш	173	123	Ш
134	92	э	174	124	Э
135	93	щ	175	125	Щ